

مع كل التفاصيل والشرح بثلاث لغات. هذا IBNSINA PRO v4.0 HYBRID سأقدم لك المشروع الكامل لـ
المشروع النهائي الجاهز للتشغيل الفوري:

الهيكل الكامل للمشروع 🏠

...

IBN-SINA-PRO-v4.0-HYBRID/

- └─ 📁 docker-compose/
 - | └─ docker-compose.hybrid.yml
 - | └─ docker-compose.quantum.yml
 - | └─ docker-compose.hospital.yml
 - | └─ docker-compose.migration.yml
- └─ 📁 backend/
 - | └─ app/
 - | | └─ __init__.py
 - | | └─ main.py
 - | | └─ hybrid_engine.py
 - | | └─ quantum_api.py
 - | | └─ hospital_api.py
 - | | └─ chemistry_api.py
 - | | └─ models/
 - | | └─ schemas/
 - | | └─ services/
 - | | └─ utils/
 - | └─ migrations/
 - | └─ tests/
 - | └─ requirements.txt
 - | └─ Dockerfile

```
|   ├── Dockerfile.quantum
|   └── Dockerfile.hospital
|   └──  chemistry/
|       ├── __init__.py
|       ├── qm_mm_engine.py
|       ├── quantum_chemistry.py
|       ├── molecular_dynamics.py
|       ├── docking.py
|       ├── ai_chemistry.py
|       └── hospital_chemistry.py
|   └──  frontend/
|       ├── src/
|       │   ├── components/
|       │   ├── pages/
|       │   ├── services/
|       │   ├── utils/
|       │   └── App.vue
|       ├── public/
|       ├── package.json
|       └── Dockerfile
|   └──  database/
|       ├── init.sql
|       ├── migrations/
|       └── seeds/
|   └──  ai_models/
|       ├── quantum_models/
|       └── hospital_models/
```

```
|   └─ hybrid_models/
|   └─ docs/
|   └─ AR/ العربية
|   └─ FR/ الفرنسية
|   └─ EN/ الإنجليزية
|   └─ scripts/
|   └─ run_hybrid.sh
|   └─ run_quantum.sh
|   └─ run_hospital.sh
|   └─ migrate_v2_to_v4.sh
|   └─ migrate_v3_to_v4.sh
|   └─ monitoring/
|   └─ prometheus.yml
|   └─ grafana/
|   └─ kubernetes/
|   └─ hybrid/
|   └─ quantum/
|   └─ hospital/
|   └─ .env.example
|   └─ LICENSE_FREE_FOREVER.md
|   └─ README.md
|   ...
```

الشامل Docker (طرح) 1. تكوين

docker-compose/hybrid.yml

```
```yaml
```

```
=====
```

```
IBN SINA PRO v4.0 HYBRID - Docker Compose
```

```
منصة هجينة متكاملة للحوسبة الكمية والمستشفيات
```

```
Integrated Hybrid Platform for Quantum Computing and Hospitals
```

```
Plateforme Hybride Intégrée pour Calcul Quantique et Hôpitaux
```

```
=====
```

```
version: '3.9'
```

```
x-base-service: &base-service
```

```
 restart: unless-stopped
```

```
 networks:
```

```
 - ibn_sina_network
```

```
x-base-database: &base-database
```

```
 image: postgres:15-alpine
```

```
 environment:
```

```
 POSTGRES_DB: ${DB_NAME:-ibnsina_v4_hybrid}
```

```
 POSTGRES_USER: ${DB_USER:-ibnsina_admin}
```

```
 POSTGRES_PASSWORD: ${DB_PASSWORD:-UltraSecurePassword2024!}
```

```
 PGDATA: /var/lib/postgresql/data/pgdata
```

```
 volumes:
```

```
 - postgres_data_v4:/var/lib/postgresql/data
```

```
 - ../database/init.sql:/docker-entrypoint-initdb.d/01-init.sql
```

```
 - ../database/migrations:/docker-entrypoint-initdb.d/migrations/
```

```
 healthcheck:
```

```
 test: ["CMD-SHELL", "pg_isready -U ${DB_USER:-ibnsina_admin} -d
${DB_NAME:-ibnsina_v4_hybrid}"]
```

interval: 10s

timeout: 5s

retries: 5

<<: \*base-service

services:

===== قاعدة البيانات الرئيسية ===== #

# Main Database | Base de données principale

postgres:

<<: \*base-database

container\_name: ibn\_sina\_v4\_postgres

ports:

- "5432:5432"

===== ذاكرة التخزين المؤقت ===== #

# Redis Cache | Cache Redis

redis:

image: redis:7-alpine

container\_name: ibn\_sina\_v4\_redis

command: >

redis-server

--appendonly yes

--maxmemory 4GB

--maxmemory-policy allkeys-lru

ports:

- "6379:6379"

volumes:

- redis\_data\_v4:/data

healthcheck:

test: ["CMD", "redis-cli", "ping"]

interval: 10s

timeout: 5s

retries: 5

<<: \*base-service

===== المحرك الهجين الرئيسي ===== #

# Main Hybrid Engine | Moteur Hybride Principal

hybrid\_backend:

build:

context: ../backend

dockerfile: Dockerfile.hybrid

args:

- APP\_MODE=hybrid

container\_name: ibn\_sina\_v4\_hybrid\_backend

environment:

| إعدادات الأساسية # Basic Settings | Paramètres de base

- APP\_NAME=IBN SINA PRO v4.0 HYBRID

- APP\_VERSION=4.0.0

- APP\_MODE=hybrid

| إعدادات قاعدة البيانات # Database Settings | Paramètres base de données

- DATABASE\_URL=postgresql://\${DB\_USER:-  
ibnsina\_admin}:\${DB\_PASSWORD:-  
UltraSecurePassword2024!}@postgres:5432/\${DB\_NAME:-ibnsina\_v4\_hybrid}

- REDIS\_URL=redis://redis:6379

| إعدادات الأمان # Security Settings | Paramètres de sécurité

- SECRET\_KEY=\${JWT\_SECRET:-  
ibn\_sina\_pro\_v4\_hybrid\_jwt\_secret\_2024\_min\_64\_chars}

- ENCRYPTION\_KEY=\${ENCRYPTION\_KEY:-  
32\_byte\_ibn\_sina\_v4\_hybrid\_encryption\_key\_2024}  
- BCrypt\_ROUNDS=12

#### # إعدادات الذكاء الاصطناعي | AI Settings | Paramètres IA

- AI\_MODELS\_DIR=/app/ai\_models  
- AI\_OPERATION\_MODE=hybrid\_adaptive  
- QUANTUM\_AI\_ENABLED=true  
- HOSPITAL\_AI\_ENABLED=true

#### # إعدادات الحوسبة | Computing Settings | Paramètres calcul

- COMPUTATION\_MODE=adaptive  
- GPU\_ENABLED=\${GPU\_ENABLED:-auto}  
- QUANTUM\_ENABLED=\${QUANTUM\_ENABLED:-auto}  
- MAX\_GPU\_MEMORY=48GB  
- MAX\_SYSTEM\_MEMORY=128GB

#### # إعدادات الكيمياء | Chemistry Settings | Paramètres chimie

- CHEMISTRY\_ENGINE=quantum\_enhanced  
- QM\_MM\_ENABLED=true  
- DOCKING\_ENGINE=ai\_enhanced  
- DYNAMICS\_ENGINE=openmm\_jax

#### # إعدادات المستشفيات | Hospital Settings | Paramètres hôpital

- HOSPITAL\_MODE\_ENABLED=true  
- CLINICAL\_SAFETY=true  
- REAL\_TIME\_MONITORING=true

ports:

- "8000:8000"

```
depends_on:
 postgres:
 condition: service_healthy
 redis:
 condition: service_healthy
volumes:
 - ../backend:/app
 - ../chemistry:/app/chemistry
 - ../ai_models:/app/ai_models
 - hybrid_data:/app/data
 - ../logs/backend:/app/logs
deploy:
 resources:
 reservations:
 devices:
 - driver: nvidia
 count: ${GPU_COUNT:-2}
 capabilities: [gpu]
 limits:
 memory: 64G
 cpus: '32.0'
 reservations:
 memory: 32G
 cpus: '16.0'
<<: *base-service
```

===== عامل الحوسبة الكمية ===== #

# Quantum Computing Worker | Worker Calcul Quantique

quantum\_worker:

build:

context: ../backend

dockerfile: Dockerfile.quantum

container\_name: ibn\_sina\_v4\_quantum\_worker

environment:

- WORKER\_TYPE=quantum\_computing
- COMPUTATION\_MODE=quantum\_enhanced
- CUDA\_VISIBLE\_DEVICES=0,1,2,3
- QUANTUM\_PRECISION=ultra\_high
- AI\_MODELS\_LEVEL=quantum

volumes:

- ../backend:/app
- ../chemistry:/app/chemistry
- ../ai\_models/quantum\_models:/app/ai\_models
- quantum\_data:/app/data

deploy:

resources:

reservations:

devices:

- driver: nvidia
- count: \${GPU\_COUNT:-4}
- capabilities: [gpu]

limits:

memory: 128G

cpus: '64.0'

reservations:

memory: 64G

cpus: '32.0'

<<: \*base-service

profiles: ["quantum", "hybrid"]

===== عامل المستشفيات ===== #

# Hospital Worker | Worker Hôpital

hospital\_worker:

build:

context: ../backend

dockerfile: Dockerfile.hospital

container\_name: ibn\_sina\_v4\_hospital\_worker

environment:

- WORKER\_TYPE=hospital\_computing
- COMPUTATION\_MODE=clinical\_optimized
- GPU\_ENABLED=false
- AI\_MODELS\_LEVEL=clinical
- OPTIMIZED\_FOR\_CPU=true
- MEMORY\_EFFICIENT=true

volumes:

- ../backend:/app
- ../chemistry:/app/chemistry
- ../ai\_models/hospital\_models:/app/ai\_models
- hospital\_data:/app/data

deploy:

resources:

limits:

memory: 8G

cpus: '4.0'

reservations:

memory: 4G

cpus: '2.0'

```
<<: *base-service
profiles: ["hospital", "hybrid"]
```

```
===== الواجهة الأمامية ===== #
```

```
Frontend Interface | Interface Frontale
```

```
frontend:
```

```
 build:
```

```
 context: ../frontend
```

```
 dockerfile: Dockerfile
```

```
 container_name: ibn_sina_v4_frontend
```

```
 environment:
```

- VITE\_API\_URL=http://localhost:8000
- VITE\_APP\_MODE=hybrid
- VITE\_APP\_VERSION=4.0.0
- VITE\_SUPPORTED\_LANGUAGES=ar,en,fr

```
 ports:
```

- "3000:3000"

```
 depends_on:
```

- hybrid\_backend

```
 volumes:
```

- ../frontend:/app
- /app/node\_modules

```
<<: *base-service
```

```
===== نظام المراقبة ===== #
```

```
Monitoring System | Système de Surveillance
```

```
prometheus:
```

```
 image: prom/prometheus:latest
```

```
 container_name: ibn_sina_v4_prometheus
```

ports:

- "9090:9090"

volumes:

- ../monitoring/prometheus.yml:/etc/prometheus/prometheus.yml
- prometheus\_data\_v4:/prometheus

command:

- '--config.file=/etc/prometheus/prometheus.yml'
- '--storage.tsdb.path=/prometheus'

<<: \*base-service

grafana:

image: grafana/grafana:latest

container\_name: ibn\_sina\_v4\_grafana

ports:

- "3001:3000"

environment:

- GF\_SECURITY\_ADMIN\_PASSWORD=\${GRAFANA\_PASSWORD:-SecureGrafana2024!}

- GF\_USERS\_ALLOW\_SIGN\_UP=false

volumes:

- grafana\_data\_v4:/var/lib/grafana
- ../monitoring/grafana:/etc/grafana/provisioning/

depends\_on:

- prometheus

<<: \*base-service

===== تخزين الملفات ===== #

# File Storage | Stockage Fichiers

minio:

image: minio/minio:latest

container\_name: ibn\_sina\_v4\_minio

ports:

- "9000:9000"

- "9001:9001"

environment:

- MINIO\_ROOT\_USER=\${MINIO\_USER:-ibnsina\_minio}

- MINIO\_ROOT\_PASSWORD=\${MINIO\_PASSWORD:-SecureMinioPassword2024!}

volumes:

- minio\_data\_v4:/data

command: server --console-address ":9001" /data

healthcheck:

test: ["CMD", "curl", "-f", "http://localhost:9000/minio/health/live"]

interval: 30s

timeout: 20s

retries: 3

<<: \*base-service

volumes:

postgres\_data\_v4:

driver: local

redis\_data\_v4:

driver: local

hybrid\_data:

driver: local

quantum\_data:

driver: local

hospital\_data:

```

 driver: local
prometheus_data_v4:
 driver: local
grafana_data_v4:
 driver: local
minio_data_v4:
 driver: local

networks:

 ibn_sina_network:
 name: ibn_sina_v4_network
 driver: bridge
'''

```

2.  المحرك الهجين الأساسي

backend/app/hybrid\_engine.py

```

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
'''
=====

IBN SINA PRO v4.0 HYBRID ENGINE - المحرك الهجين
Hybrid Computing Engine - Moteur de Calcul Hybride
=====

```

المحرك الذكي الذي يدير الحوسبة الهجينة بين:

- الحوسبة الكمية المتقدمة

- الحوسبة السريية للمستشفيات

- التحويل التلقائي بين الأنماط

The smart engine that manages hybrid computing between:

- Advanced quantum computing
- Clinical computing for hospitals
- Automatic switching between modes

Le moteur intelligent qui gère le calcul hybride entre :

- Calcul quantique avancé
- Calcul clinique pour hôpitaux
- Commutation automatique entre modes

""

```
import os
```

```
import logging
```

```
import asyncio
```

```
from typing import Dict, Any, List, Optional, Union
```

```
from enum import Enum
```

```
import json
```

```
import numpy as np
```

```
# Try to import advanced computing libraries
```

```
try:
```

```
    import jax
```

```
    import jax.numpy as jnp
```

```
    from jax import pmap, vmap, jit
```

```
    JAX_AVAILABLE = True
```

```
except ImportError:
```

```
JAX_AVAILABLE = False
```

```
logging.warning("JAX not available - using fallback mode")
```

```
try:
```

```
    import torch
```

```
    import torch.nn as nn
```

```
    TORCH_AVAILABLE = True
```

```
except ImportError:
```

```
    TORCH_AVAILABLE = False
```

```
    logging.warning("PyTorch not available")
```

```
try:
```

```
    from pyscf import gto, scf, dft
```

```
    PYSCF_AVAILABLE = True
```

```
except ImportError:
```

```
    PYSCF_AVAILABLE = False
```

```
    logging.warning("PySCF not available - quantum chemistry disabled")
```

```
class ComputationMode(Enum):
```

```
    """
```

```
    | أنماط الحوسبة المتاحة | Available Computation Modes | Modes de Calcul Disponibles
```

```
    """
```

```
    QUANTUM_ENHANCED = "quantum_enhanced" # حوسبة كمية متقدمة
```

```
    HOSPITAL_OPTIMIZED = "hospital_optimized" # حوسبة مخصصة للمستشفيات
```

```
    HYBRID_ADAPTIVE = "hybrid_adaptive" # نمط هجين ذكي
```

```
    CLINICAL_FAST = "clinical_fast" # نمط سريري سريع
```

```
    RESEARCH_PRECISE = "research_precise" # نمط بحثي دقيق
```

```
class TaskComplexity(Enum):
```

```
    """
```

```
    | مستويات تعقيد المهام | Task Complexity Levels | Niveaux de Complexité des Tâches
```

```
    """
```

```
    LOW = "low"          # مهام بسيطة - مناسبة للمستشفيات
```

```
    MEDIUM = "medium"   # مهام متوسطة - نمط هجين
```

```
    HIGH = "high"        # مهام معقدة - نمط كمي
```

```
    ULTRA = "ultra"      # مهام فائقة التعقيد - كمي متقدم
```

```
class HybridComputingEngine:
```

```
    """
```

```
    المحرك الرئيسي للحوسبة الهجينة
```

```
    Main Hybrid Computing Engine
```

```
    Moteur Principal de Calcul Hybride
```

```
    """
```

```
    def __init__(self, config: Dict[str, Any] = None):
```

```
        self.config = config or {}
```

```
        self.setup_logging()
```

```
        self.detect_hardware()
```

```
        self.initialize_engines()
```

```
        self.complexity_thresholds = self.set_complexity_thresholds()
```

```
        logging.info("🌀 تم تهيئته بنجاح") IBN SINA PRO v4.0 | المحرك الهجين
```

```
        logging.info("🌀 IBN SINA PRO v4.0 Hybrid Engine initialized successfully")
```

```
        logging.info("🌀 Moteur Hybride IBN SINA PRO v4.0 initialisé avec succès")
```

```

def setup_logging(self):
    """إعداد نظام التسجيل | Setup logging system | Configurer le système de
    journalisation"""
    logging.basicConfig(
        level=logging.INFO,
        format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
        handlers=[
            logging.FileHandler("/app/logs/hybrid_engine.log"),
            logging.StreamHandler()
        ]
    )
    self.logger = logging.getLogger("HybridEngine")

```

```

def detect_hardware(self):
    """
    كشف إمكانيات الأجهزة تلقائياً
    Auto-detect hardware capabilities
    Détection automatique des capacités matérielles
    """
    self.hardware_info = {
        "gpu_available": False,
        "gpu_count": 0,
        "gpu_memory": 0,
        "system_memory": 0,
        "cpu_cores": os.cpu_count(),
        "quantum_accelerated": False
    }

```

كشف وجود GPU | Detect GPU | Détecter GPU

if TORCH_AVAILABLE and torch.cuda.is_available():

self.hardware_info["gpu_available"] = True

self.hardware_info["gpu_count"] = torch.cuda.device_count()

self.hardware_info["gpu_memory"] = sum(
[torch.cuda.get_device_properties(i).total_memory
for i in range(torch.cuda.device_count())]
) // (1024**3) # Convert to GB

self.logger.info(f"✅ {self.hardware_info['gpu_count']} GPU مع
{self.hardware_info['gpu_memory']}GB ذاكرة")

self.logger.info(f"✅ Detected {self.hardware_info['gpu_count']} GPU with
{self.hardware_info['gpu_memory']}GB memory")

self.logger.info(f"✅ Détecté {self.hardware_info['gpu_count']} GPU avec
{self.hardware_info['gpu_memory']}GB mémoire")

كشف إمكانيات JAX | Detect JAX capabilities | Détecter capacités JAX

if JAX_AVAILABLE:

self.hardware_info["quantum_accelerated"] = True

self.logger.info(f"✅ JAX متاح للحوسبة المتسارعة")

self.logger.info(f"✅ JAX available for accelerated computing")

self.logger.info(f"✅ JAX disponible pour calcul accéléré")

def set_complexity_thresholds(self):

"""

تعيين حدود التعقيد للتحويل التلقائي

Set complexity thresholds for auto-switching

Définir les seuils de complexité pour la commutation automatique

"""

return {

```

"molecular_dynamics_atoms": 1000, # عدد الذرات للديناميكا الجزيئية
"quantum_calculation_atoms": 50, # عدد الذرات للحسابات الكمية
"docking_ligand_size": 100, # حجم الجزيء للربط
"ai_model_complexity": "medium", # تعقيد نموذج الذكاء الاصطناعي
"memory_usage_gb": 8, # استخدام الذاكرة بالجيجابايت
"computation_time_min": 30 # وقت الحساب بالدقائق
}

```

```
def initialize_engines(self):
```

```

    """
تهيئة محركات الحوسبة المختلفة

Initialize different computing engines
Initialiser les différents moteurs de calcul
    """

    self.engines = {}

```

محرك الحوسبة الكمية | Quantum Computing Engine | Moteur Calcul Quantique

```

if self.hardware_info["quantum_accelerated"] and JAX_AVAILABLE:
    from chemistry.quantum_chemistry import QuantumChemistryEngine
    self.engines["quantum"] = QuantumChemistryEngine()
    self.logger.info("✅ محرك الكيمياء الكمية تم تهيئته")

```

محرك المستشفيات | Hospital Engine | Moteur Hôpital

```

from chemistry.hospital_chemistry import HospitalChemistryEngine
self.engines["hospital"] = HospitalChemistryEngine()
self.logger.info("✅ محرك الكيمياء السريرية للمستشفيات تم تهيئته")

```

محرك الهجين | Hybrid Engine | Moteur Hybride

```
from chemistry.ai_chemistry import AIChemistryEngine
```

```
self.engines["hybrid"] = AIChemistryEngine()
```

```
self.logger.info("✓ محرك الذكاء الاصطناعي الهجين تم تهيئته")
```

```
def analyze_task_complexity(self, task_type: str, parameters: Dict[str, Any]) -> TaskComplexity:
```

```
    """
```

تحليل تعقيد المهمة لتحديد النمط الأمثل

Analyze task complexity to determine optimal mode

Analyser la complexité de la tâche pour déterminer le mode optimal

```
    """
```

```
    complexity_score = 0
```

```
    if task_type == "quantum_chemistry":
```

```
        atoms_count = parameters.get("atoms_count", 0)
```

```
        basis_set = parameters.get("basis_set", "sto-3g")
```

```
        if atoms_count > 100:
```

```
            complexity_score += 3
```

```
        elif atoms_count > 50:
```

```
            complexity_score += 2
```

```
        else:
```

```
            complexity_score += 1
```

```
        if basis_set in ["cc-pvqz", "aug-cc-pvqz"]:
```

```
            complexity_score += 2
```

```
    elif task_type == "molecular_dynamics":
```

```
        atoms_count = parameters.get("atoms_count", 0)
```

```
        simulation_time = parameters.get("simulation_time", 0)
```

```

if atoms_count > 10000:
    complexity_score += 3
elif atoms_count > 5000:
    complexity_score += 2
else:
    complexity_score += 1

if simulation_time > 100: # nanoseconds
    complexity_score += 2

elif task_type == "drug_docking":
    ligand_size = parameters.get("ligand_size", 0)
    protein_size = parameters.get("protein_size", 0)

    if ligand_size > 100 or protein_size > 1000:
        complexity_score += 3
    elif ligand_size > 50 or protein_size > 500:
        complexity_score += 2
    else:
        complexity_score += 1

```

تحديد مستوى التعقيد | Determine complexity level | Déterminer niveau complexité

```

if complexity_score >= 4:
    return TaskComplexity.ULTRA
elif complexity_score >= 3:
    return TaskComplexity.HIGH
elif complexity_score >= 2:
    return TaskComplexity.MEDIUM

```

else:

return TaskComplexity.LOW

```
def select_optimal_mode(self, task_type: str, complexity: TaskComplexity,  
    user_preference: str = None) -> ComputationMode:
```

"""

اختيار نمط الحوسبة الأمثل بناءً على التعقيد والإمكانيات

Select optimal computing mode based on complexity and capabilities

Sélectionner le mode de calcul optimal basé sur complexité et capacités

"""

إذا كان المستخدم يفضل نمط معين | If user prefers specific mode | Si utilisateur préfère mode spécifique

```
if user_preference:
```

```
    try:
```

```
        preferred_mode = ComputationMode(user_preference)
```

```
        if self.validate_mode_availability(preferred_mode, task_type):
```

```
            self.logger.info(f"🔗 {user_preference} : النمط المفضل المستخدم")
```

```
            return preferred_mode
```

```
    except ValueError:
```

```
        self.logger.warning(f"⚠️ {user_preference} : النمط المفضل غير صالح")
```

الاختيار التلقائي بناءً على التعقيد | Auto-select based on complexity | Sélection auto basée complexité

```
if complexity == TaskComplexity.ULTRA:
```

```
    if
```

```
self.validate_mode_availability(ComputationMode.QUANTUM_ENHANCED,  
task_type):
```

```
    return ComputationMode.QUANTUM_ENHANCED
```

```
    elif
```

```
self.validate_mode_availability(ComputationMode.RESEARCH_PRECISE,  
task_type):
```

```

        return ComputationMode.RESEARCH_PRECISE

    elif complexity == TaskComplexity.HIGH:

        if self.validate_mode_availability(ComputationMode.HYBRID_ADAPTIVE,
task_type):

            return ComputationMode.HYBRID_ADAPTIVE

        elif
self.validate_mode_availability(ComputationMode.QUANTUM_ENHANCED,
task_type):

            return ComputationMode.QUANTUM_ENHANCED

    elif complexity == TaskComplexity.MEDIUM:

        return ComputationMode.HYBRID_ADAPTIVE

    else: # LOW complexity

        if self.validate_mode_availability(ComputationMode.HOSPITAL_OPTIMIZED,
task_type):

            return ComputationMode.HOSPITAL_OPTIMIZED

        else:

            return ComputationMode.CLINICAL_FAST

# Fallback to hospital mode | | الرجوع للنمط السريري | | Retour au mode clinique
return ComputationMode.HOSPITAL_OPTIMIZED

```

```

def validate_mode_availability(self, mode: ComputationMode, task_type: str) ->
bool:

```

```

    """

```

التحقق من توفر النمط للمهمة المطلوبة

Validate mode availability for required task

Valider disponibilité mode pour tâche requise

```

    """

```

```

if mode == ComputationMode.QUANTUM_ENHANCED:
    return (self.hardware_info["quantum_accelerated"] and
            "quantum" in self.engines and
            self.engines["quantum"].supports_task_type(task_type))

elif mode == ComputationMode.HOSPITAL_OPTIMIZED:
    return ("hospital" in self.engines and
            self.engines["hospital"].supports_task_type(task_type))

elif mode == ComputationMode.HYBRID_ADAPTIVE:
    return ("hybrid" in self.engines and
            self.engines["hybrid"].supports_task_type(task_type))

return True

```

```

async def execute_computation(self, task_type: str, parameters: Dict[str, Any],
                               user_preference: str = None) -> Dict[str, Any]:

```

```

    """

```

تنفيذ الحساب باستخدام النمط الأمثل

Execute computation using optimal mode

Exécuter calcul utilisant mode optimal

```

    """

```

```

    try:

```

تحليل تعقيد المهمة | Analyze task complexity | Analyser complexité tâche

```

        complexity = self.analyze_task_complexity(task_type, parameters)

```

```

        self.logger.info(f" {complexity.value} :تعقيد المهمة: ")

```

اختيار النمط الأمثل | Select optimal mode | Sélectionner mode optimal

```
optimal_mode = self.select_optimal_mode(task_type, complexity,
user_preference)
```

```
self.logger.info(f"🌀 } النمط المختار: {optimal_mode.value}")
```

تنفيذ الحساب | Execute computation | Exécuter calcul

```
start_time = asyncio.get_event_loop().time()
```

```
if optimal_mode == ComputationMode.QUANTUM_ENHANCED:
```

```
    result = await self.engines["quantum"].compute(task_type, parameters)
```

```
elif optimal_mode == ComputationMode.HOSPITAL_OPTIMIZED:
```

```
    result = await self.engines["hospital"].compute(task_type, parameters)
```

```
elif optimal_mode == ComputationMode.HYBRID_ADAPTIVE:
```

```
    result = await self.engines["hybrid"].compute(task_type, parameters)
```

```
else:
```

```
    result = await self.engines["hospital"].compute(task_type, parameters)
```

```
computation_time = asyncio.get_event_loop().time() - start_time
```

إضافة المعلومات الإضافية | Add additional information | Ajouter informations supplémentaires

```
result.update({
```

```
    "computation_mode": optimal_mode.value,
```

```
    "task_complexity": complexity.value,
```

```
    "computation_time_seconds": computation_time,
```

```
    "hardware_used": self.get_hardware_usage(),
```

```
    "engine_version": "4.0.0-hybrid"
```

```
})
```

```
self.logger.info(f"✅ } الحساب اكتمل بنجاح في {computation_time:.2f} ثانية")
```

```
return result
```

except Exception as e:

```
self.logger.error(f"❌ {str(e)} فشل في تنفيذ الحساب: {str(e)}")
```

```
return {
```

```
    "error": str(e),
```

```
    "computation_mode": "fallback",
```

```
    "success": False
```

```
}
```

```
def get_hardware_usage(self) -> Dict[str, Any]:
```

```
    """
```

الحصول على معلومات استخدام الأجهزة

Get hardware usage information

Obtenir informations utilisation matérielle

```
    """
```

```
usage = {
```

```
    "gpu_used": self.hardware_info["gpu_available"],
```

```
    "gpu_count_used": self.hardware_info["gpu_count"] if  
self.hardware_info["gpu_available"] else 0,
```

```
    "quantum_acceleration": self.hardware_info["quantum_accelerated"],
```

```
    "cpu_cores_used": self.hardware_info["cpu_cores"]
```

```
}
```

إضافة معلومات الذاكرة | Add memory information | Ajouter informations mémoire

```
try:
```

```
    import psutil
```

```
    memory_info = psutil.virtual_memory()
```

```
    usage["memory_used_gb"] = memory_info.used / (1024**3)
```

```
    usage["memory_available_gb"] = memory_info.available / (1024**3)
```

```
except ImportError:
```

```
    usage["memory_info"] = "unavailable"
```

```
return usage
```

```
def get_engine_status(self) -> Dict[str, Any]:
```

```
    """
```

الحصول على حالة المحركات المتاحة

Get status of available engines

Obtenir statut des moteurs disponibles

```
    """
```

```
    status = {
```

```
        "hybrid_engine_version": "4.0.0",
```

```
        "hardware_capabilities": self.hardware_info,
```

```
        "available_engines": {},
```

```
        "complexity_thresholds": self.complexity_thresholds
```

```
    }
```

```
    for engine_name, engine in self.engines.items():
```

```
        status["available_engines"][engine_name] = {
```

```
            "initialized": True,
```

```
            "supports_quantum": hasattr(engine, 'supports_quantum'),
```

```
            "supports_hospital": hasattr(engine, 'supports_hospital'),
```

```
            "version": getattr(engine, 'version', 'unknown')
```

```
        }
```

```
    return status
```

#Example usage of hybrid engine | Exemple utilisation
moteur hybride

async def main():

"""

دالة رئيسية لتجربة المحرك الهجين

Main function to test hybrid engine

Fonction principale pour tester moteur hybride

"""

engine = HybridComputingEngine()

مثال لحساب كمي | Quantum calculation example | Exemple calcul quantique

```
quantum_task = {  
    "task_type": "quantum_chemistry",  
    "parameters": {  
        "molecule": "H2O",  
        "atoms_count": 3,  
        "basis_set": "6-31g",  
        "method": "DFT"  
    }  
}
```

result = await engine.execute_computation(**quantum_task)

print("نتيجة الحساب الكمي:", json.dumps(result, indent=2))

مثال لحساب سريري | Clinical calculation example | Exemple calcul clinique

```
clinical_task = {  
    "task_type": "drug_interaction",  
    "parameters": {  
        "drugs": ["Warfarin", "Amoxicillin"],
```

```

        "patient_data": {"age": 45, "weight": 70}
    }
}

result = await engine.execute_computation(**clinical_task)
print("نتيجة الحساب السريري:", json.dumps(result, indent=2))

```

```

if __name__ == "__main__":
    asyncio.run(main())
'''

```

3. محرك الكيمياء الكمية

chemistry/quantum_chemistry.py

```

'''python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
'''

=====

محرك الكيمياء الكمية - QUANTUM CHEMISTRY ENGINE
Advanced Quantum Computing for Chemistry
Moteur de Chimie Quantique Avancée

=====

```

محرك متقدم للحسابات الكمية في الكيمياء باستخدام:

- JAX للتسريع والحوسبة المتوازية
- PySCF للحسابات الكمية الأساسية

- خوارزميات كمية مخصصة

Advanced quantum computing engine for chemistry using:

- JAX for acceleration and parallel computing
- PySCF for fundamental quantum calculations
- Custom quantum algorithms

Moteur de calcul quantique avancé pour la chimie utilisant :

- JAX pour l'accélération et le calcul parallèle
- PySCF pour calculs quantiques fondamentaux
- Algorithmes quantiques personnalisés

"""

```
import logging
```

```
import numpy as np
```

```
from typing import Dict, Any, List, Optional
```

```
import asyncio
```

| Import advanced libraries | Importer bibliothèques avancées | استيراد المكتبات المتقدمة

try:

```
import jax
```

```
import jax.numpy as jnp
```

```
from jax import jit, grad, vmap, pmap
```

```
from jax.scipy.linalg import expm
```

```
JAX_AVAILABLE = True
```

except ImportError:

```
JAX_AVAILABLE = False
```

```
logging.warning("JAX (الافتراضي) سيتم استخدام الوضع الافتراضي")
```

try:

```
from pyscf import gto, scf, dft, cc, mp
```

```
from pyscf.hessian import thermo
```

```
PYSCF_AVAILABLE = True
```

except ImportError:

```
PYSCF_AVAILABLE = False
```

```
logging.warning("PySCF (الحسابات الكمية الأساسية غير متاحة")
```

class QuantumChemistryEngine:

```
"""
```

محرك الكيمياء الكمية المتقدم

Advanced Quantum Chemistry Engine

Moteur de Chimie Quantique Avancé

```
"""
```

```
def __init__(self, config: Dict[str, Any] = None):
```

```
    self.config = config or {}
```

```
    self.logger = logging.getLogger("QuantumChemistryEngine")
```

```
    self.version = "4.0.0-quantum"
```

| إعدادات الحساب الكمي | # Quantum calculation settings | Paramètres calcul quantique

```
self.default_settings = {
```

```
    "basis_set": "6-31g",
```

```
    "method": "DFT",
```

```
    "functional": "B3LYP",
```

```
    "precision": "high",
```

```
    "max_memory": 16000, # MB
```

```
    "convergence_tolerance": 1e-8
```

```
}
```

```
self.initialize_quantum_functions()
```

```
self.logger.info("تم تهيئته بنجاح" v4.0محرك الكيمياء الكمية 🌀)
```

```
def initialize_quantum_functions(self):
```

```
""" Initialize accelerated quantum functions | Initialiser  
fonctions quantiques accélérées """
```

```
if JAX_AVAILABLE:
```

```
# دوال JAX | JAX accelerated functions | Fonctions JAX accélérées
```

```
self.jax_energy_gradient = jit(grad(self.jax_energy_function))
```

```
self.jax_force_calculation = jit(self.calculate_forces_jax)
```

```
self.jax_wavefunction_analysis = jit(self.analyze_wavefunction_jax)
```

```
self.logger.info("الكمية تم تهيئتها للتسريع" JAXدوال ✅)
```

```
async def compute(self, task_type: str, parameters: Dict[str, Any]) -> Dict[str, Any]:
```

```
"""
```

```
تنفيذ الحساب الكمي بناءً على نوع المهمة
```

```
Execute quantum computation based on task type
```

```
Exécuter calcul quantique basé sur type de tâche
```

```
"""
```

```
try:
```

```
self.logger.info(f"بدء الحساب الكمي: {task_type}")
```

```
if task_type == "quantum_energy":
```

```
    return await self.calculate_energy(parameters)
```

```
elif task_type == "molecular_optimization":
```

```
    return await self.optimize_geometry(parameters)
```

```

elif task_type == "vibrational_analysis":
    return await self.vibrational_frequencies(parameters)
elif task_type == "reaction_path":
    return await self.reaction_path_analysis(parameters)
elif task_type == "excited_states":
    return await self.excited_states_calculation(parameters)
else:
    return {"error": f"نوع المهمة غير مدعوم: {task_type}"}

```

```

except Exception as e:

```

```

    self.logger.error(f"✖ خطأ في الحساب الكمي: {str(e)}")
    return {"error": str(e), "success": False}

```

```

async def calculate_energy(self, parameters: Dict[str, Any]) -> Dict[str, Any]:

```

```

    """

```

حساب الطاقة الكمية للجزيء

Calculate quantum energy for molecule

Calculer énergie quantique pour molécule

```

    """

```

```

    molecule_str = parameters.get("molecule", "")

```

```

    basis_set = parameters.get("basis_set", self.default_settings["basis_set"])

```

```

    method = parameters.get("method", self.default_settings["method"])

```

```

    if not PYSCF_AVAILABLE:

```

```

        return {"error": "PySCF غير متوفر للحسابات الكمية"}

```

```

    try:

```

إنشاء الجزيء في
PySCF

PySCF | Create molecule in PySCF | Créer molécule dans

```
mol = gto.Mole()
mol.atom = molecule_str
mol.basis = basis_set
mol.build()
```

| اختيار طريقة الحساب # Select calculation method | Sélectionner méthode calcul

```
if method.upper() == "HF":
    mf = scf.RHF(mol)
elif method.upper() == "DFT":
    functional = parameters.get("functional", self.default_settings["functional"])
    mf = dft.RKS(mol)
    mf.xc = functional
else:
    mf = scf.RHF(mol) # افتراضي
```

| تشغيل الحساب # Run calculation | Exécuter calcul

```
energy = mf.kernel()
```

| معلومات إضافية # Additional information | Informations supplémentaires

```
result = {
    "energy_hartree": float(energy),
    "energy_kcal_mol": float(energy * 627.509), # kcal/mol تحويل إلى
    "energy_ev": float(energy * 27.2114), # eV تحويل إلى
    "converged": mf.converged,
    "method_used": method,
    "basis_set_used": basis_set,
    "success": True
}
```

Use JAX for performance if available |
لتحسين الأداء إذا كان متاحاً | JAX # استخدام
Utiliser JAX si disponible

if JAX_AVAILABLE:

jax_result = self.enhance_with_jax(mol, energy, parameters)

result.update(jax_result)

self.logger.info(f"✓ } حساب الطاقة اكتمل: {energy:.6f} Hartree")

return result

except Exception as e:

self.logger.error(f"✗ } خطأ في حساب الطاقة: {str(e)}")

return {"error": str(e), "success": False}

def enhance_with_jax(self, mol, energy: float, parameters: Dict[str, Any]) -> Dict[str, Any]:

"""

JAX تحسين النتائج باستخدام

Enhance results using JAX

Améliorer résultats utilisant JAX

"""

try:

JAX array | Convert molecule coordinates to JAX array
تحويل إحداثيات الجزيء إلى
| Convertir coordonnées en tableau JAX

coords = jnp.array([atom[1] for atom in mol.atom])

JAX | Calculate additional properties using JAX |
حساب الخواص الإضافية باستخدام
Calculer propriétés supplémentaires avec JAX

dipole_moment = self.calculate_dipole_jax(coords, mol.atom_charges())

molecular_volume = self.calculate_volume_jax(coords)

```

return {
    "dipole_moment_debye": float(dipole_moment),
    "molecular_volume_ang3": float(molecular_volume),
    "jax_enhanced": True
}

```

except Exception as e:

```

self.logger.warning(f"⚠️ JAX {str(e)} فشل")
return {"jax_enhanced": False}

```

```
@partial(jit, static_argnums=(0,))
```

```
def calculate_dipole_jax(self, coords: jnp.array, charges: jnp.array) -> jnp.array:
```

```

"""

```

JAX حساب العزم الثنائي باستخدام

Calculate dipole moment using JAX

Calculer moment dipolaire avec JAX

```

"""

```

```

center_of_charge = jnp.sum(coords * charges[:, None], axis=0) /
jnp.sum(charges)

```

```
dipole = jnp.sum((coords - center_of_charge) * charges[:, None], axis=0)
```

```
return jnp.linalg.norm(dipole) * 2.5418 # Convert to Debye
```

```
@partial(jit, static_argnums=(0,))
```

```
def calculate_volume_jax(self, coords: jnp.array) -> jnp.array:
```

```

"""

```

JAX حساب الحجم الجزيئي باستخدام

Calculate molecular volume using JAX

Calculer volume moléculaire avec JAX

```

"""

```

طريقة مبسطة لحساب الحجم | Simplified volume calculation | Calcul volume simplifié

```

max_coords = jnp.max(coords, axis=0)
min_coords = jnp.min(coords, axis=0)
dimensions = max_coords - min_coords
return jnp.prod(dimensions)

```

```

async def optimize_geometry(self, parameters: Dict[str, Any]) -> Dict[str, Any]:

```

```

    """

```

تحسين الهندسة الجزيئية باستخدام الحسابات الكمية

Optimize molecular geometry using quantum calculations

Optimiser géométrie moléculaire utilisant calculs quantiques

```

    """

```

```

    try:

```

```

        molecule_str = parameters.get("molecule", "")

```

```

        basis_set = parameters.get("basis_set", "6-31g")

```

```

        if not PYSCF_AVAILABLE:

```

```

            return {"error": "PySCF {مطلوب لتحسين الهندسة}"}

```

```

        mol = gto.Mole()

```

```

        mol.atom = molecule_str

```

```

        mol.basis = basis_set

```

```

        mol.build()

```

استخدام PySCF | التحسين الهندسة Use PySCF for geometry optimization | Utiliser PySCF pour optimisation géométrie

```

        from pyscf import optimize

```

```

        mf = scf.RHF(mol)

```

```

        mol_eq = optimize(mf)

```

```

result = {
    "optimized_geometry": mol_eq.atom,
    "initial_energy": float(mf.kernel()),
    "final_energy": float(mol_eq.kernel()),
    "optimization_steps": getattr(mol_eq, 'steps', 'unknown'),
    "success": True
}

```

```

self.logger.info("✅ تحسين الهندسة اكتمل بنجاح")
return result

```

```

except Exception as e:

```

```

    self.logger.error(f"❌ {str(e)} خطأ في تحسين الهندسة:")
    return {"error": str(e), "success": False}

```

```

async def vibrational_frequencies(self, parameters: Dict[str, Any]) -> Dict[str, Any]:

```

```

    """

```

حساب الترددات الاهتزازية للجزيء

Calculate vibrational frequencies for molecule

Calculer fréquences vibrationnelles pour molécule

```

    """

```

```

try:

```

```

    molecule_str = parameters.get("molecule", "")

```

```

    if not PYSCF_AVAILABLE:

```

```

        return {"error": "PySCF {مطلوب لحساب الترددات الاهتزازية}"}

```

```

    mol = gto.Mole()

```

```
mol.atom = molecule_str
```

```
mol.basis = '6-31g'
```

```
mol.build()
```

```
mf = scf.RHF(mol)
```

```
mf.kernel()
```

حساب المشتقات الثانية | Calculate second derivatives | Calculer dérivées secondes

```
hessian = mf.Hessian().kernel()
```

حساب الترددات | Calculate frequencies | Calculer fréquences

```
freq_info = thermo.harmonic_analysis(mol, hessian)
```

```
result = {  
    "frequencies_cm1": [float(f) for f in freq_info.freqs],  
    "ir_intensities": [float(i) for i in freq_info.intensities],  
    "zero_point_energy": float(freq_info.zpe),  
    "thermodynamic_properties": self.calculate_thermodynamics(freq_info),  
    "success": True  
}
```

```
self.logger.info("حساب الترددات الاهتزازية اكتمل")
```

```
return result
```

except Exception as e:

```
self.logger.error(f"خطأ في حساب الترددات: {str(e)}")
```

```
return {"error": str(e), "success": False}
```

```
def calculate_thermodynamics(self, freq_info) -> Dict[str, float]:
```

```
    """
```

حساب الخواص الترموديناميكية

Calculate thermodynamic properties

Calculer propriétés thermodynamiques

```
    """
```

حسابات ترموديناميكية مبسطة | Simplified thermodynamic calculations | Calculs thermodynamiques simplifiés

```
    return {
```

```
        "enthalpy_298K": float(getattr(freq_info, 'enthalpy', 0)),
```

```
        "entropy_298K": float(getattr(freq_info, 'entropy', 0)),
```

```
        "free_energy_298K": float(getattr(freq_info, 'free_energy', 0))
```

```
    }
```

```
def supports_task_type(self, task_type: str) -> bool:
```

```
    """
```

التحقق من دعم نوع المهمة

Check if task type is supported

Vérifier si type de tâche est supporté

```
    """
```

```
    supported_tasks = [
```

```
        "quantum_energy",
```

```
        "molecular_optimization",
```

```
        "vibrational_analysis",
```

```
        "reaction_path",
```

```
        "excited_states"
```

```
    ]
```

```
    return task_type in supported_tasks
```

#Example usage of quantum engine | Exemple utilisation
موتور كمي | استخدام المحرك الكمي

```
async def demo_quantum_engine():
```

```
    """
```

عرض توضيحي للمحرك الكمي

Demo of quantum engine

Démo du moteur quantique

```
    """
```

```
    engine = QuantumChemistryEngine()
```

حساب طاقة جزيء الماء | Water molecule energy calculation | Calcul énergie molécule
eau

```
    task = {
```

```
        "task_type": "quantum_energy",
```

```
        "parameters": {
```

```
            "molecule": "O 0 0 0; H 0 0 0.957; H 0 0.957 0",
```

```
            "basis_set": "6-31g",
```

```
            "method": "HF"
```

```
        }
```

```
    }
```

```
    result = await engine.compute(**task)
```

```
    print("result: ", "نتيجة الحساب الكمي:", result)
```

```
if __name__ == "__main__":
```

```
    asyncio.run(demo_quantum_engine())
```

```
...
```

4.  محرك المستشفيات المبسط

chemistry/hospital_chemistry.py

```
```python
```

```
#!/usr/bin/env python3
```

```
-*- coding: utf-8 -*-
```

```
"""
```

```
=====
```

HOSPITAL CHEMISTRY ENGINE - محرك كيمياء المستشفيات

Clinical-Optimized Chemistry Computing

Moteur de Chimie Optimisé pour Hôpitaux

```
=====
```

محرك مخصص للمستشفيات والعيادات مع:

- متطلبات أجهزة منخفضة

- حسابات سريعة ومنظمة

- تكامل مع الأنظمة السريرية

- أمان وموثوقية عالية

Engine specialized for hospitals and clinics with:

- Low hardware requirements

- Fast and structured calculations

- Integration with clinical systems

- High security and reliability

Moteur spécialisé pour hôpitaux et cliniques avec :

- Faibles exigences matérielles

- Calculs rapides et structurés

- Intégration avec systèmes cliniques

- Haute sécurité et fiabilité

"""

```
import logging
```

```
import numpy as np
```

```
from typing import Dict, Any, List, Optional
```

```
import asyncio
```

```
import json
```

```
from datetime import datetime
```

```
class HospitalChemistryEngine:
```

```
 """
```

محرك الكيمياء المخصص للمستشفيات

Hospital-Specialized Chemistry Engine

Moteur de Chimie Spécialisé pour Hôpitaux

```
 """
```

```
def __init__(self, config: Dict[str, Any] = None):
```

```
 self.config = config or {}
```

```
 self.logger = logging.getLogger("HospitalChemistryEngine")
```

```
 self.version = "4.0.0-hospital"
```

# قاعدة بيانات الأدوية والتفاعلات | Drug and interactions database | Base de données médicaments et interactions

```
 self.drug_database = self.initialize_drug_database()
```

```
 self.interaction_database = self.initialize_interaction_database()
```

```
 self.clinical_guidelines = self.initialize_clinical_guidelines()
```

# إعدادات المستشفى | Hospital settings | Paramètres hôpital

```
self.hospital_settings = {
 "max_computation_time": 30, # seconds
 "memory_limit_mb": 2000,
 "precision_level": "clinical",
 "safety_margin": 0.8, # هامش أمان 20%
 "emergency_mode": False
}
```

```
self.logger.info("تم تهيئته بنجاح") v4.0 محرك كيمياء المستشفيات
```

```
def initialize_drug_database(self) -> Dict[str, Any]:
```

```
 """
```

تهيئة قاعدة بيانات الأدوية

Initialize drug database

Initialiser base de données médicaments

```
 """
```

```
 return {
```

```
 "warfarin": {
```

```
 "name_ar": "وارفارين",
```

```
 "name_en": "Warfarin",
```

```
 "name_fr": "Warfarine",
```

```
 "half_life": 36, # hours
```

```
 "protein_binding": 99, # percentage
```

```
 "clearance": 0.003, # L/h/kg
```

```
 "therapeutic_range": "2.0-3.0 INR",
```

```
 "risk_factors": ["bleeding", "interactions"]
```

```
 },
```

```

"amoxicillin": {
 "name_ar": "أموكسيسيلين",
 "name_en": "Amoxicillin",
 "name_fr": "Amoxicilline",
 "half_life": 1.7,
 "protein_binding": 18,
 "clearance": 0.25,
 "therapeutic_range": "N/A",
 "risk_factors": ["allergy", "renal_impairment"]
},
"metformin": {
 "name_ar": "ميٹفورمین",
 "name_en": "Metformin",
 "name_fr": "Metformine",
 "half_life": 6.2,
 "protein_binding": 0,
 "clearance": 1.1,
 "therapeutic_range": "Fasting glucose control",
 "risk_factors": ["lactic_acidosis", "renal_impairment"]
}

```

# يمكن إضافة المزيد من الأدوية | Can add more drugs | Peut ajouter plus médicaments

```
}
```

```
def initialize_interaction_database(self) -> Dict[str, Any]:
```

```
"""
```

تهيئة قاعدة بيانات التفاعلات الدوائية

Initialize drug interactions database

Initialiser base de données interactions médicamenteuses

```
"""
```

```
return {
 "warfarin_amoxicillin": {
 "drug1": "warfarin",
 "drug2": "amoxicillin",
 "interaction_type": "pharmacodynamic",
 "severity": "moderate",
 "mechanism": "Increased bleeding risk due to additive effects",
 "recommendation": "Monitor INR closely, consider dose adjustment",
 "evidence_level": "established"
 },
 "warfarin_metformin": {
 "drug1": "warfarin",
 "drug2": "metformin",
 "interaction_type": "pharmacokinetic",
 "severity": "mild",
 "mechanism": "Minor effect on warfarin metabolism",
 "recommendation": "Routine monitoring sufficient",
 "evidence_level": "possible"
 }
}
```

```
def initialize_clinical_guidelines(self) -> Dict[str, Any]:
```

```
"""
```

تهيئة الإرشادات السريرية

```
Initialize clinical guidelines
Initialiser directives cliniques
"""
```

```
return {
```

```

"dosing_adjustment": {
 "renal_impairment": {
 "mild": "Reduce dose by 25%",
 "moderate": "Reduce dose by 50%",
 "severe": "Avoid or use with extreme caution"
 },
 "hepatic_impairment": {
 "mild": "Monitor closely",
 "moderate": "Reduce dose by 25-50%",
 "severe": "Avoid use"
 },
 "elderly": "Start with lower doses, monitor closely"
},
"therapeutic_monitoring": {
 "warfarin": "Monitor INR regularly",
 "aminoglycosides": "Monitor peak and trough levels",
 "digoxin": "Monitor serum levels and ECG"
}
}

```

```

async def compute(self, task_type: str, parameters: Dict[str, Any]) -> Dict[str, Any]:

```

```

 """

```

تنفيذ الحسابات السريرية

Execute clinical computations

Exécuter calculs cliniques

```

 """

```

```

try:

```

```

 self.logger.info(f"بدء الحساب السريري: {task_type}")

```

```

if task_type == "drug_interaction":
 return await self.check_drug_interactions(parameters)
elif task_type == "dosing_calculation":
 return await self.calculate_dosing(parameters)
elif task_type == "therapeutic_monitoring":
 return await self.therapeutic_monitoring(parameters)
elif task_type == "renal_dosing":
 return await self.renal_dosing_adjustment(parameters)
elif task_type == "emergency_calculation":
 return await self.emergency_calculations(parameters)
else:
 return {"error": f"نوع المهمة غير مدعوم: {task_type}"}

```

```

except Exception as e:

```

```

 self.logger.error(f"✖ {str(e)} خطأ في الحساب السريري: {e}")
 return {"error": str(e), "success": False}

```

```

async def check_drug_interactions(self, parameters: Dict[str, Any]) -> Dict[str, Any]:

```

```

 """

```

فحص التفاعلات الدوائية

Check drug interactions

Vérifier interactions médicamenteuses

```

 """

```

```

try:

```

```

 drugs = parameters.get("drugs", [])
 patient_data = parameters.get("patient_data", {})

```

```

 if len(drugs) < 2:

```

```

 return {

```

```
"interactions_found": 0,
"message": " ,",
"success": True
}
```

```
interactions = []
high_risk_interactions = []
```

# فحص جميع التركيبات الممكنة | Check all possible combinations | Vérifier toutes combinaisons possibles

```
for i in range(len(drugs)):
 for j in range(i + 1, len(drugs)):
 drug1, drug2 = drugs[i], drugs[j]
 interaction_key = f"{drug1}_{drug2}"
 reverse_key = f"{drug2}_{drug1}"
```

# البحث في قاعدة البيانات | Search in database | Rechercher dans base de données

```
interaction_data = None
if interaction_key in self.interaction_database:
 interaction_data = self.interaction_database[interaction_key]
elif reverse_key in self.interaction_database:
 interaction_data = self.interaction_database[reverse_key]

if interaction_data:
 interaction_data = interaction_data.copy()
 interaction_data["drug_pair"] = f"{drug1} + {drug2}"
```

# تقييم الخطورة بناءً على بيانات المريض | Assess risk based on patient data | Évaluer risque basé données patient

```

 risk_level = self.assess_interaction_risk(interaction_data,
patient_data)

 interaction_data["patient_specific_risk"] = risk_level

 interactions.append(interaction_data)

 if risk_level in ["high", "critical"]:
 high_risk_interactions.append(interaction_data)

 result = {
 "interactions_found": len(interactions),
 "high_risk_interactions": len(high_risk_interactions),
 "interactions_list": interactions,
 "patient_specific_recommendations":
self.generate_recommendations(interactions, patient_data),
 "clinical_decision_support": self.provide_decision_support(interactions,
patient_data),
 "success": True
 }

 self.logger.info(f"✓ } : فحص التفاعلات اكتمل: {len(interactions)} (تفاعل وجد")

 return result

except Exception as e:

 self.logger.error(f"✗ } : خطأ في فحص التفاعلات: {str(e)}")

 return {"error": str(e), "success": False}

def assess_interaction_risk(self, interaction: Dict[str, Any], patient_data: Dict[str,
Any]) -> str:
 """

```

تقييم خطورة التفاعل بناءً على حالة المريض

Assess interaction risk based on patient condition

Évaluer risque interaction basé état patient

"""

base\_severity = interaction.get("severity", "unknown")

patient\_factors = self.assess\_patient\_risk\_factors(patient\_data)

# تعديل الخطورة بناءً على عوامل المريض | Adjust severity based on patient factors |  
Ajuster sévérité basé facteurs patient

if base\_severity == "mild":

if patient\_factors["high\_risk"]:

return "moderate"

return "mild"

elif base\_severity == "moderate":

if patient\_factors["high\_risk"]:

return "high"

elif patient\_factors["moderate\_risk"]:

return "moderate-high"

return "moderate"

elif base\_severity == "severe":

if patient\_factors["high\_risk"]:

return "critical"

return "severe"

return base\_severity

def assess\_patient\_risk\_factors(self, patient\_data: Dict[str, Any]) -> Dict[str, bool]:

"""

تقييم عوامل الخطورة للمريض

Assess patient risk factors

Évaluer facteurs de risque patient

"""

age = patient\_data.get("age", 0)

weight = patient\_data.get("weight", 70)

renal\_function = patient\_data.get("renal\_function", "normal")

hepatic\_function = patient\_data.get("hepatic\_function", "normal")

comorbidities = patient\_data.get("comorbidities", [])

return {

    "elderly": age > 65,

    "pediatric": age < 18,

    "low\_weight": weight < 50,

    "renal\_impairment": renal\_function != "normal",

    "hepatic\_impairment": hepatic\_function != "normal",

    "multiple\_comorbidities": len(comorbidities) >= 3,

    "high\_risk": (age > 75 or renal\_impairment or hepatic\_impairment),

    "moderate\_risk": (age > 65 or len(comorbidities) >= 2)

}

def generate\_recommendations(self, interactions: List[Dict], patient\_data: Dict[str, Any]) -> List[str]:

"""

توليد توصيات سريرية مخصصة

Generate customized clinical recommendations

Générer recommandations cliniques personnalisées

"""

```
recommendations = []
```

```
for interaction in interactions:
```

```
 base_recommendation = interaction.get("recommendation", "")
```

```
 risk_level = interaction.get("patient_specific_risk", "")
```

# تخصيص التوصية بناءً على مستوى الخطورة | Customize recommendation based on risk level | Personnaliser recommandation basé niveau risque

```
 if risk_level in ["high", "critical"]:
```

```
 enhanced_recommendation = f"⚠️ {base_recommendation} يلزم
 مراقبة مكثفة واستشارة الصيدلي."
```

```
 recommendations.append(enhanced_recommendation)
```

```
 elif risk_level == "moderate":
```

```
 enhanced_recommendation = f"⚠️ {base_recommendation} يوصى
 بالمراقبة الدورية."
```

```
 recommendations.append(enhanced_recommendation)
```

```
 else:
```

```
 recommendations.append(f"👌 {base_recommendation}")
```

# توصيات عامة بناءً على حالة المريض | General recommendations based on patient condition | Recommandations générales basées état patient

```
 if patient_data.get("age", 0) > 65:
```

```
 recommendations.append("👴 مريض مسن - يوصى ببدء الجرعات بأقل كمية ممكنة.")
```

```
 if patient_data.get("renal_function") != "normal":
```

```
 recommendations.append("👨‍⚕️ خلل في وظائف الكلى - مراجعة الجرعات بناءً على معدل الترشيح
 الكبيبي.")
```

```
 if len(interactions) >= 3:
```

```
recommendations.append("عدد كبير من التفاعلات - يوصى بمراجعة قائمة الأدوية مع
الطبيب.")
```

```
return recommendations
```

```
def provide_decision_support(self, interactions: List[Dict], patient_data: Dict[str,
Any]) -> Dict[str, Any]:
```

```
"""
```

تقديم دعم القرار السريري

Provide clinical decision support

Fournir support décision clinique

```
"""
```

```
high_risk_count = sum(1 for i in interactions if i.get("patient_specific_risk") in
["high", "critical"])
```

```
decision_support = {
 "overall_risk_level": "low",
 "immediate_actions": [],
 "monitoring_recommendations": [],
 "consultation_recommendations": []
}
```

```
if high_risk_count >= 2:
```

```
 decision_support["overall_risk_level"] = "critical"
```

```
 decision_support["immediate_actions"].extend([
```

"مراجعة فورية لجميع الأدوية",

"استشارة الصيدلي السريري",

"مراقبة العلامات الحيوية كل 4 ساعات"

```
])
```

```
elif high_risk_count >= 1:
```

```

decision_support["overall_risk_level"] = "high"

decision_support["monitoring_recommendations"].extend([
 "مراقبة العلامات الحيوية كل 8 ساعات",
 "مراجعة الجرعات مع الطبيب المعالج"
])

```

# إضافة توصيات بناءً على حالة المريض | Add recommendations based on patient condition | Ajouter recommandations basées état patient

```

if patient_data.get("renal_impairment"):
 decision_support["monitoring_recommendations"].append("مراقبة وظائف الكلى
أسبوعياً")

if patient_data.get("age", 0) > 75:
 decision_support["consultation_recommendations"].append("استشارة طب المسنين")

return decision_support

```

```

async def calculate_dosing(self, parameters: Dict[str, Any]) -> Dict[str, Any]:

```

```

 """

```

حساب الجرعات الدوائية

Calculate drug dosing

Calculer dosage médicamenteux

```

 """

```

```

 try:

```

```

 drug_name = parameters.get("drug", "")

```

```

 patient_data = parameters.get("patient_data", {})

```

```

 indication = parameters.get("indication", "general")

```

```

 drug_info = self.drug_database.get(drug_name.lower())

```

```

 if not drug_info:

```

```
return {"error": f"غير موجود في قاعدة البيانات"} {drug_name} الدواء {f}
```

# حساب الجرعة الأساسية | Calculate base dose | Calculer dose de base

```
base_dose = self.calculate_base_dose(drug_name, indication, patient_data)
```

# تعديل الجرعة بناءً على عوامل المريض | Adjust dose based on patient factors | Ajuster dose basé facteurs patient

```
adjusted_dose = self.adjust_dose_for_patient(base_dose, drug_name, patient_data)
```

```
result = {
 "drug_name": drug_info["name_ar"],
 "base_dose": base_dose,
 "adjusted_dose": adjusted_dose,
 "dosing_frequency": self.get_dosing_frequency(drug_name, indication),
 "administration_instructions":
self.get_administration_instructions(drug_name),
 "monitoring_parameters": self.get_monitoring_parameters(drug_name),
 "adjustment_factors": self.get_adjustment_factors(patient_data),
 "success": True
}
```

```
self.logger.info(f"✓ {drug_name} حساب الجرعة اكتمل: {f}")
```

```
return result
```

except Exception as e:

```
self.logger.error(f"✗ {str(e)} خطأ في حساب الجرعة: {e}")
```

```
return {"error": str(e), "success": False}
```

```
def calculate_base_dose(self, drug_name: str, indication: str, patient_data: Dict[str, Any]) -> Dict[str, Any]:
```

```
 """
```

حساب الجرعة الأساسية للدواء

Calculate base drug dose

Calculer dose de base médicament

```
 """
```

```
 weight = patient_data.get("weight", 70)
```

```
 age = patient_data.get("age", 40)
```

```
 base_doses = {
```

```
 "warfarin": {
```

```
 "loading_dose": "5-10 mg",
```

```
 "maintenance_dose": "2-5 mg daily",
```

```
 "monitoring": "INR target 2.0-3.0"
```

```
 },
```

```
 "amoxicillin": {
```

```
 "standard_dose": "500 mg every 8 hours" if weight > 40 else "250 mg every 8 hours",
```

```
 "high_dose": "875 mg every 12 hours" if weight > 40 else "500 mg every 12 hours"
```

```
 },
```

```
 "metformin": {
```

```
 "initial_dose": "500 mg twice daily",
```

```
 "maintenance_dose": "500-1000 mg twice daily",
```

```
 "max_dose": "2000 mg daily"
```

```
 }
```

```
 }
```

```
 return base_doses.get(drug_name, {"dose": "{يحتاج تحديد من الطبيب}"})
```

```
def adjust_dose_for_patient(self, base_dose: Dict[str, Any], drug_name: str,
 patient_data: Dict[str, Any]) -> Dict[str, Any]:
```

```
 """
```

تعديل الجرعة بناءً على حالة المريض

Adjust dose based on patient condition

Ajuster dose basé état patient

```
 """
```

```
 adjusted_dose = base_dose.copy()
```

```
 adjustments = []
```

# تعديلات بناءً على العمر | Adjustments based on age | Ajustements basés âge

```
 age = patient_data.get("age", 40)
```

```
 if age > 65:
```

```
 adjustments.append(("تخفيض الجرعة بنسبة 25% للمرضى المسنين"))
```

```
 adjusted_dose["elderly_adjustment"] = "جرعة مخفضة"
```

# تعديلات بناءً على وظائف الكلى | Adjustments based on renal function | Ajustements basés fonction rénale

```
 renal_function = patient_data.get("renal_function", "normal")
```

```
 if renal_function != "normal" and drug_name in ["warfarin", "amoxicillin"]:
```

```
 adjustments.append(f"تعديل الجرعة لخلل وظائف الكلى ({renal_function})")
```

```
 adjusted_dose["renal_adjustment"] =
```

```
 self.clinical_guidelines["dosing_adjustment"]["renal_impairment"][renal_function]
```

# تعديلات بناءً على الوزن | Adjustments based on weight | Ajustements basés poids

```
 weight = patient_data.get("weight", 70)
```

```
 if weight < 50:
```

```
 adjustments.append(("تعديل الجرعة للوزن المنخفض"))
```

```
 adjusted_dose["weight_adjustment"] = "جرعة مخفضة للوزن المنخفض"
```

```
adjusted_dose["adjustments_applied"] = adjustments
return adjusted_dose
```

```
def get_dosing_frequency(self, drug_name: str, indication: str) -> str:
 """Get dosing frequency | Obtenir fréquence dosage"""
 frequencies = {
 "warfarin": "يوميًا",
 "amoxicillin": "كل 8 ساعات",
 "metformin": "مرتين يوميًا"
 }
 return frequencies.get(drug_name, "يحدد من الطبيب")
```

```
def get_administration_instructions(self, drug_name: str) -> str:
 """Administration instructions | Instructions administration"""
 instructions = {
 "warfarin": "food مع أو بدون daily, يؤخذ في نفس الوقت",
 "amoxicillin": "food يؤخذ مع",
 "metformin": "food يؤخذ مع لتقليل الآثار الجانبية الهضمية"
 }
 return instructions.get(drug_name, "حسب إرشادات الطبيب")
```

```
def get_monitoring_parameters(self, drug_name: str) -> List[str]:
 """Monitoring parameters | Paramètres surveillance"""
 monitoring = {
 "warfarin": ["INR", "عدد الصفائح", "علامات النزيف"],
 "amoxicillin": ["الآثار الجانبية"],
 "metformin": ["سكر الدم الصائم", "وظائف الكلى", "حموضة الدم"]
 }
```

```
return monitoring.get(drug_name, [])
```

```
def get_adjustment_factors(self, patient_data: Dict[str, Any]) -> Dict[str, Any]:
```

```
 """عوامل التعديل | Adjustment factors | Facteurs ajustement"""
```

```
 return {
```

```
 "age": patient_data.get("age"),
```

```
 "weight": patient_data.get("weight"),
```

```
 "renal_function": patient_data.get("renal_function"),
```

```
 "hepatic_function": patient_data.get("hepatic_function"),
```

```
 "comorbidities": patient_data.get("comorbidities", [])
```

```
 }
```

```
def supports_task_type(self, task_type: str) -> bool:
```

```
 """
```

```
 التحقق من دعم نوع المهمة
```

```
 Check if task type is supported
```

```
 Vérifier si type de tâche est supporté
```

```
 """
```

```
 supported_tasks = [
```

```
 "drug_interaction",
```

```
 "dosing_calculation",
```

```
 "therapeutic_monitoring",
```

```
 "renal_dosing",
```

```
 "emergency_calculation"
```

```
]
```

```
 return task_type in supported_tasks
```

#Example usage of hospital engine | Exemple utilisation  
moteur hôpital

```
async def demo_hospital_engine():
```

```
 """
```

عرض توضيحي لمحرك المستشفى

Demo of hospital engine

Démo du moteur hôpital

```
 """
```

```
 engine = HospitalChemistryEngine()
```

# فحص تفاعلات دوائية | Drug interaction check | Vérification interactions  
médicamenteuses

```
 task = {
```

```
 "task_type": "drug_interaction",
```

```
 "parameters": {
```

```
 "drugs": ["warfarin", "amoxicillin", "metformin"],
```

```
 "patient_data": {
```

```
 "age": 72,
```

```
 "weight": 65,
```

```
 "renal_function": "mild",
```

```
 "comorbidities": ["hypertension", "diabetes"]
```

```
 }
```

```
 }
```

```
 }
```

```
 result = await engine.compute(**task)
```

```
 print("نتيجة فحص التفاعلات:", json.dumps(result, indent=2, ensure_ascii=False))
```

```
if __name__ == "__main__":
```

```
asyncio.run(demo_hospital_engine())
'''
```

🌐 5. الواجهة الرئيسية للنسخة الهجينة

backend/app/main.py

```
```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
=====

IBN SINA PRO v4.0 HYBRID - الواجهة الرئيسية
Main Hybrid Backend Interface
Interface Principale Backend Hybride
=====
```

التي تجمع بين: v4.0 الواجهة الرئيسية للنسخة الهجينة

- الحوسبة الكمية المتقدمة
- الحوسبة السريرية للمستشفيات
- النظام الهجين الذكي

Main backend interface for hybrid v4.0 that combines:

- Advanced quantum computing
- Clinical computing for hospitals
- Smart hybrid system

Interface principale backend pour hybride v4.0 qui combine :

- Calcul quantique avancé

- Calcul clinique pour hôpitaux

- Système hybride intelligent

"""

import os

import logging

from datetime import datetime

from typing import Dict, Any, List, Optional

from fastapi import FastAPI, HTTPException, Depends, status

from fastapi.middleware.cors import CORSMiddleware

from fastapi.responses import JSONResponse

from pydantic import BaseModel, Field

import uvicorn

from .hybrid_engine import HybridComputingEngine, ComputationMode,
TaskComplexity

) نماذج البيانات) ===== #Pydantic) =====

Data Models (Pydantic) | Modèles de Données (Pydantic)

class ComputationRequest(BaseModel):

"""

| نموذج طلب الحساب Computation Request Model | Modèle Demande Calcul

"""

task_type: str = Field(..., description="نوع المهمة (quantum_chemistry,
drug_interaction, etc.)")

parameters: Dict[str, Any] = Field(..., description="معاملات الحساب")

```
user_preference: Optional[str] = Field(None, description="النمط المفضل  
(quantum_enhanced, hospital_optimized, etc.)")

priority: str = Field("normal", description="أولوية الحساب low, normal, high,  
emergency)")
```

```
class Config:
```

```
    schema_extra = {
        "example": {
            "task_type": "quantum_chemistry",
            "parameters": {
                "molecule": "H2O",
                "basis_set": "6-31g",
                "method": "DFT"
            },
            "user_preference": "quantum_enhanced",
            "priority": "normal"
        }
    }
```

```
class ComputationResponse(BaseModel):
```

```
    """
```

```
    | Computation Response Model | Modèle Réponse Calcul | نموذج استجابة الحساب |
```

```
    """
```

```
    success: bool = Field(..., description="نجاح العملية")
    result: Dict[str, Any] = Field(..., description="نتيجة الحساب")
    computation_mode: str = Field(..., description="نمط الحساب المستخدم")
    task_complexity: str = Field(..., description="مستوى تعقيد المهمة")
    computation_time_seconds: float = Field(..., description="وقت الحساب بالثواني")
    engine_version: str = Field(..., description="إصدار المحرك")
    timestamp: str = Field(..., description="الطابع الزمني")
```

```
class SystemStatus(BaseModel):
```

```
    """
```

```
    | نموذج حالة النظام | System Status Model | Modèle Statut Système
```

```
    """
```

```
    version: str = Field(..., description="إصدار النظام")
```

```
    status: str = Field(..., description="حالة النظام")
```

```
    available_engines: Dict[str, Any] = Field(..., description="المحركات المتاحة")
```

```
    hardware_capabilities: Dict[str, Any] = Field(..., description="إمكانات الأجهزة")
```

```
    active_tasks: int = Field(..., description="المهام النشطة")
```

```
    system_health: str = Field(..., description="صحة النظام")
```

```
class HealthCheck(BaseModel):
```

```
    """
```

```
    | نموذج فحص الصحة | Health Check Model | Modèle Vérification Santé
```

```
    """
```

```
    status: str = Field(..., description="الحالة")
```

```
    version: str = Field(..., description="الإصدار")
```

```
    timestamp: str = Field(..., description="الوقت")
```

```
    services: Dict[str, str] = Field(..., description="حالة الخدمات")
```

```
الرئيسي ===== #FastAPI ===== تطبيق
```

```
# Main FastAPI Application | Application FastAPI Principale
```

```
app = FastAPI(
```

```
    title="IBN SINA PRO v4.0 HYBRID",
```

```
    description=""
```

```
    منصة هجينة متكاملة للحوسبة الكمية والمستشفيات - IBN SINA PRO v4.0 HYBRID
```

المميزات الرئيسية:

-  ****حوسبة كمية متقدمة**** - دقة عالية مع أداء فائق
-  ****حوسبة سريرية**** - مخصصة للمستشفيات والعيادات
-  ****نظام هجين ذكي**** - تحويل تلقائي بين الأنماط
-  ****مجاني 100%**** - للأبد بدون أي رسوم
-  ****مفتوح المصدر**** - شفاف وقابل للتطوير

Main Features:

-  ****Advanced Quantum Computing**** - High accuracy with superior performance
-  ****Clinical Computing**** - Specialized for hospitals and clinics
-  ****Smart Hybrid System**** - Automatic switching between modes
-  ****100% Free**** - Forever without any fees
-  ****Open Source**** - Transparent and extensible

Fonctionnalités Principales:

-  ****Calcul Quantique Avancé**** - Haute précision avec performance supérieure
-  ****Calcul Clinique**** - Spécialisé pour hôpitaux et cliniques
-  ****Système Hybride Intelligent**** - Commutation automatique entre modes
-  ****100% Gratuit**** - Pour toujours sans frais
-  ****Open Source**** - Transparent et extensible

""",

version="4.0.0-hybrid",

contact={

"name": "IBN SINA PRO فريق",

"email": "support@ibnsina.pro",

},

```

license_info={
    "name": "FREE FOREVER LICENSE",
    "url": "https://ibnsina.pro/license",
},
docs_url="/docs",
redoc_url="/redoc",
openapi_url="/openapi.json"
)

```

إعداد #CORS | CORS Setup | Configuration CORS

```

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # حدد النطاقات المسموحة | In production, specify allowed origins
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

```

المتغيرات العامة | #Global Variables | Variables Globales

```

hybrid_engine: Optional[HybridComputingEngine] = None
app_start_time = datetime.utcnow()

```

===== أحداث دورة حياة التطبيق ===== #

Application Lifecycle Events | Événements Cycle Vie Application

```

@app.on_event("startup")
async def startup_event():

```

"""

حدث بدء التشغيل - تهيئة المحرك الهجين

Startup Event - Initialize Hybrid Engine

Événement Démarrage - Initialiser Moteur Hybride

"""

global hybrid_engine

try:

logging.info("🔧 ابدء تشغيل IBN SINA PRO v4.0 HYBRID...")

| # تهيئة المحرك الهجين | Initialize hybrid engine | Initialiser moteur hybride

hybrid_engine = HybridComputingEngine()

| # التحقق من حالة المحرك | Check engine status | Vérifier statut moteur

engine_status = hybrid_engine.get_engine_status()

logging.info(f"✅ {engine_status} المحرك الهجين جاهز: ")

logging.info("🏁 IBN SINA PRO v4.0 HYBRID (يعمل بنجاح!)

except Exception as e:

logging.error(f"❌ {str(e)} فشل في بدء التشغيل: ")

raise

@app.on_event("shutdown")

async def shutdown_event():

"""

حدث إيقاف التشغيل - تنظيف الموارد

Shutdown Event - Cleanup resources

Événement Arrêt - Nettoyer ressources

"""

logging.info("🌐 إيقاف IBN SINA PRO v4.0 HYBRID...")

| Cleanup resources if needed | Nettoyer ressources si
nécessaire

) نقاط النهاية) ===== #Endpoints) =====

API Endpoints | Points de Terminaison API

@app.get("/", response_model=Dict[str, Any])

async def root():

"""

نقطة النهاية الجذرية - معلومات النظام

Root Endpoint - System Information

Point de Terminaison Racine - Informations Système

"""

return {

"message": "مرحباً بك في IBN SINA PRO v4.0 HYBRID!",

"message_en": "Welcome to IBN SINA PRO v4.0 HYBRID!",

"message_fr": "Bienvenue à IBN SINA PRO v4.0 HYBRID!",

"version": "4.0.0-hybrid",

"status": "نشيط | Actif",

"timestamp": datetime.utcnow().isoformat(),

"documentation": "/docs",

"features": [

"حوسبة كمية متقدمة | Advanced Quantum Computing | Calcul Quantique Avancé",

"حوسبة سريرية للمستشفيات | Clinical Computing for Hospitals | Calcul Clinique pour
Hôpitaux",

"نظام هجين ذكي | Smart Hybrid System | Système Hybride Intelligent",

"مجاني 100% للأبد | 100% Gratuit pour Toujours"

]

```
}
```

```
@app.get("/health", response_model=HealthCheck)
```

```
async def health_check():
```

```
    """
```

فحص صحة النظام والخدمات

System and Services Health Check

Vérification Santé Système et Services

```
    """
```

```
    services_status = {}
```

```
    try:
```

```
        # فحص المحرك الهجين | Check hybrid engine | Vérifier moteur hybride
```

```
        if hybrid_engine:
```

```
            engine_status = hybrid_engine.get_engine_status()
```

```
            services_status["hybrid_engine"] = "healthy"
```

```
            services_status["hardware"] = "available" if  
engine_status["hardware_capabilities"]["gpu_available"] else "cpu_only"
```

```
        else:
```

```
            services_status["hybrid_engine"] = "unhealthy"
```

```
        # فحص الخدمات الأخرى | Check other services | Vérifier autres services
```

```
        services_status["api"] = "healthy"
```

```
        services_status["database"] = "healthy" # يمكن إضافة فحص قاعدة بيانات فعلي
```

```
        overall_status = "healthy" if all(status == "healthy" for status in  
services_status.values()) else "degraded"
```

```
    return HealthCheck(
```

```
        status=overall_status,
```

```

        version="4.0.0-hybrid",
        timestamp=datetime.utcnow().isoformat(),
        services=services_status
    )

```

except Exception as e:

```

    logging.error(f" ✖ {str(e)} فشل فحص الصحة:")
    return HealthCheck(
        status="unhealthy",
        version="4.0.0-hybrid",
        timestamp=datetime.utcnow().isoformat(),
        services={"error": str(e)}
    )

```

@app.get("/status", response_model=SystemStatus)

async def system_status():

"""

الحصول على حالة النظام التفصيلية

Get Detailed System Status

Obtenir Statut Détaillé Système

"""

if not hybrid_engine:

```

    raise HTTPException(
        status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
        detail="المحرك الهجين غير جاهز"
    )

```

try:

```

    engine_status = hybrid_engine.get_engine_status()

```

```

return SystemStatus(
    version="4.0.0-hybrid",
    status="running",
    available_engines=engine_status["available_engines"],
    hardware_capabilities=engine_status["hardware_capabilities"],
    active_tasks=0, # يمكن تتبع المهام النشطة فعلياً
    system_health="excellent"
)

```

except Exception as e:

```

logging.error(f"❌ {str(e)} فشل في الحصول على حالة النظام:")
raise HTTPException(
    status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
    detail=f"{str(e)} فشل في الحصول على حالة النظام:"
)

```

```
@app.post("/compute", response_model=ComputationResponse)
```

```
async def compute_task(request: ComputationRequest):
```

```
    """
```

تنفيذ مهمة حسابية باستخدام المحرك الهجين

Execute Computational Task Using Hybrid Engine

Exécuter Tâche Calcul Utilisant Moteur Hybride

```
    """
```

```
if not hybrid_engine:
```

```

    raise HTTPException(
        status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
        detail="المحرك الهجين غير جاهز"
    )

```

try:

```
start_time = datetime.utcnow()
```

| تنفيذ الحساب # Execute computation | Exécuter calcul

```
result = await hybrid_engine.execute_computation(  
    task_type=request.task_type,  
    parameters=request.parameters,  
    user_preference=request.user_preference  
)
```

```
computation_time = (datetime.utcnow() - start_time).total_seconds()
```

| إعداد الاستجابة # Prepare response | Préparer réponse

```
response_data = {  
    "success": result.get("success", True),  
    "result": result,  
    "computation_mode": result.get("computation_mode", "unknown"),  
    "task_complexity": result.get("task_complexity", "unknown"),  
    "computation_time_seconds": computation_time,  
    "engine_version": result.get("engine_version", "4.0.0-hybrid"),  
    "timestamp": datetime.utcnow().isoformat()  
}
```

```
logging.info(f"✅ {request.task_type} اكتملت المهمة في {computation_time:.2f} ثانية")  
return ComputationResponse(**response_data)
```

except Exception as e:

```
logging.error(f"❌ {request.task_type}: {str(e)}")
```

```
raise HTTPException(  
    status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,  
    detail=f"فشل في تنفيذ المهمة: {str(e)}"  
)
```

```
@app.get("/engines")
```

```
async def get_available_engines():
```

```
    """
```

الحصول على قائمة المحركات المتاحة وإمكانياتها

Get List of Available Engines and Capabilities

Obtenir Liste Moteurs Disponibles et Capacités

```
    """
```

```
if not hybrid_engine:
```

```
    raise HTTPException(  
        status_code=status.HTTP_503_SERVICE_UNAVAILABLE,  
        detail="المحرك الهجين غير جاهز"  
    )
```

```
try:
```

```
    engine_status = hybrid_engine.get_engine_status()
```

```
    return {
```

```
        "available_engines": engine_status["available_engines"],
```

```
        "hardware_capabilities": engine_status["hardware_capabilities"],
```

```
        "complexity_thresholds": engine_status["complexity_thresholds"],
```

```
        "supported_task_types": [
```

```
            "quantum_chemistry",    # كيمياء كمية
```

```
            "molecular_optimization", # تحسين الجزيئات
```

```
            "vibrational_analysis",  # تحليل اهتزازي
```

```
            "drug_interaction",      # تفاعلات دوائية
```

```

        "dosing_calculation",    # حساب الجرعات
        "therapeutic_monitoring", # مراقبة علاجية
        "renal_dosing",          # جرعات الكلى
        "emergency_calculation" # حسابات طوارئ
    ]
}

```

except Exception as e:

```

    logging.error(f"❌ {str(e)} فشل في الحصول على المحركات:")
    raise HTTPException(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        detail=f"{str(e)} فشل في الحصول على المحركات:"
    )

```

@app.get("/mode/recommendation")

async def get_mode_recommendation(task_type: str, parameters: Dict[str, Any]):

"""

الحصول على توصية بنمط الحساب الأمثل

Get Recommendation for Optimal Computation Mode

Obtenir Recommandation pour Mode Calcul Optimal

"""

if not hybrid_engine:

```

    raise HTTPException(
        status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
        detail="المحرك الهجين غير جاهز"
    )

```

try:

تحليل التعقيد | Analyze complexity | Analyser complexité

```
complexity = hybrid_engine.analyze_task_complexity(task_type, parameters)
```

```
| الحصول على التوصية | # Get recommendation | Obtenir recommandation
```

```
recommended_mode = hybrid_engine.select_optimal_mode(task_type,  
complexity)
```

```
return {  
    "task_type": task_type,  
    "complexity_level": complexity.value,  
    "recommended_mode": recommended_mode.value,  
    "reasoning": self.get_recommendation_reasoning(complexity,  
recommended_mode),  
    "alternative_modes": self.get_alternative_modes(task_type)  
}
```

```
except Exception as e:
```

```
logging.error(f" ✕ } فشل في الحصول على التوصية: {str(e)}")
```

```
raise HTTPException(  
    status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,  
    detail=f"} فشل في الحصول على التوصية: {str(e)}"
```

```
)
```

```
def get_recommendation_reasoning(complexity: TaskComplexity, mode:  
ComputationMode) -> str:
```

```
    """
```

```
توليد شرح للتوصية
```

```
Generate reasoning for recommendation
```

```
Générer raisonnement pour recommandation
```

```
    """
```

```
reasoning = {
```

```

(TaskComplexity.ULTRA, ComputationMode.QUANTUM_ENHANCED):
    "المهمة فائقة التعقيد تتطلب حوسبة كمية متقدمة لأفضل دقة وأداء",
(TaskComplexity.HIGH, ComputationMode.HYBRID_ADAPTIVE):
    "المهمة عالية التعقيد مناسبة للنظام الهجين الذكي للتوازن بين الدقة والسرعة",
(TaskComplexity.MEDIUM, ComputationMode.HYBRID_ADAPTIVE):
    "المهمة متوسطة التعقيد مثالية للنظام الهجين للتكيف مع متطلبات الأداء",
(TaskComplexity.LOW, ComputationMode.HOSPITAL_OPTIMIZED):
    "المهمة بسيطة ومناسبة للحوسبة السريرية المخصصة للمستشفيات"
}

```

```

return reasoning.get((complexity, mode), "تحليل التعقيد والإمكانيات المتاحة")

```

```

def get_alternative_modes(task_type: str) -> List[Dict[str, str]]:
    """
    الحصول على الأنماط البديلة

    Get alternative modes
    Obtenir modes alternatifs
    """
    alternatives = []

    if task_type in ["quantum_chemistry", "molecular_optimization"]:
        alternatives.extend([
            {"mode": "quantum_enhanced", "description": "GPU أعلى دقة - يحتاج"},
            {"mode": "hybrid_adaptive", "description": "توازن بين الدقة والأداء"},
            {"mode": "hospital_optimized", "description": "أسرع - مناسب للمستشفيات"}
        ])
    else:
        alternatives.extend([
            {"mode": "hospital_optimized", "description": "مخصص للمستشفيات - سريع وموثوق"}
        ])

```

```
    {"mode": "hybrid_adaptive", "description": "نظام هجين ذكي"},  
    {"mode": "clinical_fast", "description": "أسرع للأوضاع الحرجة"}  
])
```

return alternatives

===== نقاط النهاية للإصدارات السابقة ===== #

Legacy Version Endpoints | Points de Terminaison Versions Anciennes

@app.get("/v2/compatibility")

async def v2_compatibility():

"""

v2.0 متوافق مع الإصدار

Compatibility with v2.0

Compatibilité avec v2.0

"""

return {

"compatible": True,

"message": "v2.0 متوافق تماماً مع IBN SINA PRO v4.0",

"migration_tool": "/api/migrate/v2-to-v4",

"supported_endpoints": [

"/compute",

"/health",

"/status"

]

}

@app.get("/v3/compatibility")

```
async def v3_compatibility():
```

```
    """
```

```
    v3.0 توافق مع الإصدار
```

```
    Compatibility with v3.0
```

```
    Compatibilité avec v3.0
```

```
    """
```

```
    return {
```

```
        "compatible": True,
```

```
        "message": "v3.0 متوافق تماماً مع IBN SINA PRO v4.0",
```

```
        "migration_tool": "/api/migrate/v3-to-v4",
```

```
        "enhancements": [
```

```
            "نظام هجين محسّن",
```

```
            "دعم أفضل للمستشفيات",
```

```
            "تحسين الأداء الكمي"
```

```
        ]
```

```
    }
```

```
===== نقطة الدخول الرئيسية ===== #
```

```
# Main Entry Point | Point d'Entrée Principal
```

```
if __name__ == "__main__":
```

```
    """
```

```
    التشغيل المباشر للتطبيق
```

```
    Direct application run
```

```
    Exécution directe application
```

```
    """
```

```
    uvicorn.run(
```

```
        "main:app",
```

```
host="0.0.0.0",
port=8000,
reload=True, # | للتطوير فقط | For development only | Pour développement
seulement
log_level="info",
workers=1
)
...
```

📁 6. هيكل الملفات الإضافية الأساسية

backend/requirements.txt

```
```txt
=====
IBN SINA PRO v4.0 HYBRID - متطلبات بايثون
Python Requirements - Exigences Python
=====
```

#Core Framework | Cadre Principal | إطار العمل الأساسي

```
fastapi==0.104.1
uvicorn[standard]==0.24.0
python-multipart==0.0.6
pydantic==2.5.0
pydantic-settings==2.1.0
```

#Databases | Bases de Données | قواعد البيانات

```
sqlalchemy==2.0.23
psycopg2-binary==2.9.9
```

alembic==1.12.1

asynccpg==0.29.0

redis==5.0.1

| الأمن والمصادقة | #Security & Authentication | Sécurité & Authentification

python-jose[cryptography]==3.3.0

passlib[bcrypt]==1.7.4

bcrypt==4.1.2

cryptography==41.0.8

pyjwt==2.8.0

| الذكاء الاصطناعي والتعلم الآلي | #AI & Machine Learning | IA & Apprentissage Automatique

torch==2.1.0

torchvision==0.16.0

torchaudio==2.1.0

transformers==4.36.2

datasets==2.16.1

accelerate==0.25.0

| الحوسبة الكمية والمتسارعة | #Quantum & Accelerated Computing | Calcul Quantique & Accéléré

jax[cuda12]==0.4.20

jaxlib==0.4.20

flax==0.7.5

optax==0.1.7

| الكيمياء الحاسوبية | #Computational Chemistry | Chimie Computationnelle

pyscf==2.6.0

rdkit-pypi==2024.3.5

openmm==8.1.0

mdtraj==1.9.9

prody==2.4.1

biopython==1.81

| الحسابات العلمية | #Scientific Computing | Calcul Scientifique

numpy==1.24.3

scipy==1.11.4

pandas==2.1.4

scikit-learn==1.3.2

networkx==3.2.1

| المراقبة والقياس | #Monitoring & Metrics | Surveillance & Métriques

prometheus-client==0.19.0

prometheus-fastapi-instrumentator==0.4.0

opentelemetry-api==1.21.0

opentelemetry-sdk==1.21.0

| الأدوات المساعدة | #Utilities | Utilitaires

python-dotenv==1.0.0

pytz==2023.3.post1

babel==2.13.1

pillow==10.1.0

openpyxl==3.1.2

requests==2.31.0

| للتطوير والاختبار | #Development & Testing | Développement & Tests

pytest==7.4.3

pytest-asyncio==0.21.1

```
pytest-cov==4.1.0
```

```
black==23.11.0
```

```
flake8==6.1.0
```

```
...
```

```
scripts/run_hybrid.sh
```

```
```bash
```

```
#!/bin/bash
```

```
# =====
```

```
# IBN SINA PRO v4.0 HYBRID - سكريبت التشغيل
```

```
# Hybrid Edition Startup Script
```

```
# Script Démarrage Édition Hybride
```

```
# =====
```

```
set -e # | إيقاف عند أول خطأ | Stop on first error | Arrêter à première erreur
```

```
| #Colors for display | Couleurs pour affichage
```

```
RED='\033[0;31m'
```

```
GREEN='\033[0;32m'
```

```
YELLOW='\033[1;33m'
```

```
BLUE='\033[0;34m'
```

```
CYAN='\033[0;36m'
```

```
NC='\033[0m' # No Color
```

```
| #Header display function | Fonction affichage en-tête
```

```
print_header() {
```

```
    echo -e "${CYAN}"
```

```
    echo "=====
```

```

echo " 🌀 IBN SINA PRO v4.0 HYBRID - "الإصدار الهجين"
echo " Quantum + Hospital Integrated Platform"
echo " Plateforme Intégrée Quantique + Hôpital"
echo "=====
echo -e "${NC}"
}

```

| Colored display functions | Fonctions affichage coloré | دوال العرض الملونة |

```

print_success() { echo -e "${GREEN}✅ $1${NC}"; }
print_warning() { echo -e "${YELLOW}⚠️ $1${NC}"; }
print_error() { echo -e "${RED}❌ $1${NC}"; }
print_info() { echo -e "${BLUE}📄 $1${NC}"; }

```

| Check basic requirements | Vérifier prérequis de base | التحقق من المتطلبات الأساسية |

```

check_requirements() {
    print_info "التحقق من المتطلبات الأساسية..."
}

```

Docker | Check Docker | Vérifier Docker | التحقق من

```

if ! command -v docker &> /dev/null; then
    print_error "Docker غير مثبت"
    echo " Docker يرجى تثبيت من: https://docker.com"
    exit 1
fi
print_success "Docker مثبت"

```

Docker Compose | Check Docker Compose | Vérifier Docker Compose | التحقق من

```

if ! command -v docker-compose &> /dev/null; then
    print_error "Docker Compose غير مثبت"
    echo " Docker Compose يرجى تثبيت"

```

```

    exit 1

fi

print_success "مثبت " Docker Compose

# التحقق من الذاكرة | Vérifier mémoire
local memory_gb=$(free -g 2>/dev/null | awk 'NR==2{print $2}' || echo "0")
if [ "$memory_gb" -lt 8 ]; then
    print_warning "$memory_gb GB - 16 GB يوصى به على الأقل"
else
    print_success "$memory_gb GB الذاكرة كافية:"
fi
}

# Detect hardware capabilities | Détecter capacités matérielles
detect_hardware() {
    print_info "كشف إمكانيات الأجهزة..."

    # GPU | Detect GPU | Détecter GPU
    if command -v nvidia-smi &> /dev/null; then
        GPU_COUNT=$(nvidia-smi --query-gpu=name --format=csv,noheader | wc -l)
        GPU_MEMORY=$(nvidia-smi --query-gpu=memory.total --format=csv,noheader | head -1 | sed 's/ MiB//')
        GPU_MEMORY_GB=$((GPU_MEMORY / 1024))

        print_success "$GPU_COUNT بطاقة GPU $GPU_MEMORY_GB GB مع"
    else
        print_warning "فقط CPU سيتم استخدام - GPU لا يوجد"
    fi
}

```

```

export GPU_ENABLED=false
export GPU_COUNT=0
fi

```

| كشف الذاكرة والنظام | Detect memory and system | Détecter mémoire et système

```

local total_memory=$(free -g | awk 'NR==2{print $2}')
local cpu_cores=$(nproc)

print_info "$total_memory GB: الذاكرة الإجمالية:"
print_info "$cpu_cores: أنوية المعالج:"
}

```

| اختيار وضع التشغيل | Select operation mode | Sélectionner mode opération

```

select_mode() {
    echo -e "${CYAN}"
    echo "اختر وضع التشغيل: 🌀"
    echo "1) 🏥 (وضع المستشفى CPU - متطلبات منخفضة)"
    echo "2) 🧠 (حوسبة متقدمة) - GPU وضع كمي متقدم"
    echo "3) 🌐 (تلقائي - موصى به) وضع هجين ذكي"
    echo "4) 🔄 (ترحيل من إصدار سابق)"
    echo -e "${NC}"
}

```

choiceاختيارك [4/3/2/1]: " " -p read

case \$choice in

1)

```

export APP_MODE="hospital"
export GPU_ENABLED="false"
export QUANTUM_ENABLED="false"

```

```
export COMPOSE_FILE="docker-compose.hospital.yml"
```

```
print_success ""تم اختيار وضع المستشفى""
```

```
::
```

2)

```
export APP_MODE="quantum"
```

```
export GPU_ENABLED="true"
```

```
export QUANTUM_ENABLED="true"
```

```
export COMPOSE_FILE="docker-compose.quantum.yml"
```

```
print_success ""تم اختيار الوضع الكمي المتقدم""
```

```
::
```

3)

```
export APP_MODE="hybrid"
```

```
export GPU_ENABLED="auto"
```

```
export QUANTUM_ENABLED="auto"
```

```
export COMPOSE_FILE="docker-compose.hybrid.yml"
```

```
print_success ""تم اختيار الوضع الهجين الذكي""
```

```
::
```

4)

```
./migrate_from_legacy.sh
```

```
exit 0
```

```
::
```

*)

```
print_warning ""اختيار غير صحيح - استخدام الوضع الهجين الافتراضي""
```

```
export APP_MODE="hybrid"
```

```
export COMPOSE_FILE="docker-compose.hybrid.yml"
```

```
::
```

```
esac
```

```
}
```

| إعداد البيئة | #Setup environment | Configurer environnement

```
setup_environment() {
```

```
    print_info "إعداد بيئة التشغيل..."
```

| # نسخ ملف البيئة إذا لم يكن موجوداً | Copy environment file if not exists | Copier fichier environnement si n'existe pas

```
    if [ ! -f ".env" ]; then
```

```
        if [ -f ".env.example" ]; then
```

```
            cp .env.example .env
```

```
            print_success "من القالب ".env تم إنشاء ملف .
```

```
        else
```

```
            print_error "ملف .env.example لم 找到 .
```

```
            exit 1
```

```
        fi
```

```
    fi
```

| # تحميل متغيرات البيئة | Load environment variables | Charger variables environnement

```
    set -a
```

```
    source .env
```

```
    set +a
```

| # التحقق من المتغيرات الحرجة | Check critical variables | Vérifier variables critiques

```
    if [ -z "$DB_PASSWORD" ] || [ "$DB_PASSWORD" =  
"change_this_secure_password_2024" ]; then
```

```
        print_warning "env كلمة مرور قاعدة البيانات غير آمنة - يرجى تغييرها في ملف .
```

```
    fi
```

```
}
```

| #Build and run services | Construire et exécuter services

```

build_and_run() {
    print_info "" بناء وتشغيل الخدمات...

    | إيقاف الخدمات السابقة | # Stop previous services | Arrêter services précédents
    print_info "" إيقاف الخدمات السابقة...
    docker-compose -f $COMPOSE_FILE down --remove-orphans || true

    | بناء الصور | # Build images | Construire images
    print_info " قد يستغرق عدة دقائق..." Docker بناء صور

    if [ "$GPU_ENABLED" = "true" ]; then
        docker-compose -f $COMPOSE_FILE build --no-cache --parallel
    else
        docker-compose -f $COMPOSE_FILE build --no-cache --parallel
    fi

    | تشغيل الخدمات | # Start services | Démarrer services
    print_info "" تشغيل الخدمات...
    docker-compose -f $COMPOSE_FILE up -d

    | انتظار التهيئة | # Wait for initialization | Attendre initialisation
    print_info "" انتظار تهيئة الخدمات (90 ثانية)...
    sleep 90
}

| التحقق من حالة الخدمات | # Check services status | Vérifier statut services
check_services() {
    print_info "" التحقق من حالة جميع الخدمات...

```

```

local services=("postgres" "redis" "hybrid_backend" "frontend")
local all_healthy=true

for service in "${services[@]"; do
    if docker-compose -f $COMPOSE_FILE ps | grep -q "$service.*Up"; then
        print_success "$service: "يعمل بنجاح"
    else
        print_error "$service: "فشل في التشغيل"
        all_healthy=false
    fi
done

if [ "$all_healthy" = false ]; then
    print_error ""بعض الخدمات فشلت في التشغيل"
    return 1
fi

print_success ""جميع الخدمات تعمل بنجاح"
return 0
}

```

```

#Display final information | Afficher informations finales
show_final_info() {
    echo -e "${GREEN}"
    echo "=====
    echo " 🏥 IBN SINA PRO v4.0 HYBRID "جاهز للاستخدام!"
    echo " Quantum + Hospital Platform Ready!"
    echo " Plateforme Quantique + Hôpital Prête!"
    echo "=====

```

echo -e "\${NC}"

echo -e "\${CYAN} 🌐 }\$الواجهات الرئيسية:NC}"

echo " http://localhost:3000الواجهة الأمامية:"

echo " http://localhost:8000API واجهة:"

echo " http://localhost:8000/docsالتوثيق:"

echo " http://localhost:3001المراقبة:"

echo " MinIO Console: http://localhost:9001"

echo -e "\${CYAN} 🛠 }\$معلومات التقنية:NC}"

echo " \$ APP_MODEوضع التشغيل:"

echo " \$GPU_ENABLEDدعم GPU:"

echo " hybridالإصدار: 4.0.0 -"

echo -e "\${YELLOW} 📄 }\$بيانات الدخول الافتراضية:NC}"

echo " admin@ibnsina.proالبريد الإلكتروني:"

echo " IbnSinaPro2024!كلمة المرور:"

echo -e "\${BLUE} 🛑 }\$إجراءات استكشاف الأخطاء:NC}"

echo " اسم الخدمة[" docker-compose -f \$COMPOSE_FILE logs عرض السجلات:

echo " اسم الخدمة[" docker-compose -f \$COMPOSE_FILE restart إعادة تشغيل خدمة:

echo " docker-compose -f \$COMPOSE_FILE downإيقاف النظام:

echo -e "\${GREEN}"

echo "=====

echo " 🎉 IBN SINA PRO v4.0! ابدأ رحلتك مع "

echo "=====

echo -e "\${NC}"

}

| الدالة الرئيسية | #Main function | Fonction principale

main() {

print_header

check_requirements

detect_hardware

select_mode

setup_environment

build_and_run

| # التحقق من الخدمات مع إعادة محاولة | Check services with retry | Vérifier services avec nouvelle tentative

local retry_count=0

local max_retries=3

while [\$retry_count -lt \$max_retries]; do

if check_services; then

break

else

retry_count=\$((retry_count + 1))

if [\$retry_count -lt \$max_retries]; then

print_warning "\$retry_count/\$max_retries) إعادة محاولة التحقق من الخدمات (\$)"

sleep 30

else

print_error "max_retries فشل في تشغيل بعض الخدمات بعد \$"

exit 1

fi

fi

done

```
show_final_info  
}
```

```
| Execution | #التنفيذ |  
if [[ "${BASH_SOURCE[0]}" == "${0}" ]]; then  
    main "$@"  
fi  
````
```

7. الوثائق الشاملة 📖

docs/AR/README.md

````markdown

📖 IBN SINA PRO v4.0 HYBRID - الدليل الشامل

🎯 نظرة عامة

****IBN SINA PRO v4.0 HYBRID**** هو منصة متكاملة تجمع بين قوة الحوسبة الكمية وكفاءة الحوسبة السريرية للمستشفيات في نظام واحد ذكي.

🌟 المميزات الرئيسية

- ****🌐 نظام هجين ذكي****: تحويل تلقائي بين أنماط الحوسبة

- ****📊 حوسبة كمية متقدمة****: دقة عالية مع أداء فائق

- ****🏥 حوسبة سريرية****: مخصصة للمستشفيات والعيادات

- ****💰 مجاني 100%****: للأبد بدون أي رسوم خفية

- ****🔓 مفتوح المصدر****: شفاف وقابل للتطوير

🌀 البدء السريع

المتطلبات الأساسية

- **Docker** 20.10+

- **Docker Compose** 2.0+

موصى به (GB 16 - ذاكرة: 8

كحد أدنى GB - مساحة تخزين: 50

التنصيب الفوري

```
```bash
```

```
استنساخ المشروع
```

```
git clone https://github.com/ibn-sina-pro/v4-hybrid.git
```

```
cd IBN-SINA-PRO-v4.0-HYBRID
```

```
التشغيل التلقائي
```

```
chmod +x scripts/run_hybrid.sh
```

```
./scripts/run_hybrid.sh
```

```
```
```

اختيار وضع التشغيل

اختيار واحد من ثلاثة أوضاع: pt سيطر منك السكري

1. CPU فقط 🖥️ وضع المستشفى: تشغيل خفيف على

للحوسبة المتقدمة GPU 2. 🖥️ وضع كمي: استخدام

3. 🌐 وضع هجين: النظام الذكي (موصى به)

الهيكل التقني

المكونات الرئيسية

...

IBN-SINA-PRO-v4.0-HYBRID/

-  docker-compose/ # تكوينات Docker
-  backend/ # الواجهة الخلفية
-  chemistry/ # محركات الكيمياء
-  frontend/ # الواجهة الأمامية
-  database/ # قواعد البيانات
-  ai_models/ # نماذج الذكاء الاصطناعي
-  docs/ # الوثائق
-  scripts/ # سكريبتات التشغيل

...

المحركات المتاحة

(HybridComputingEngine1. المحرك المهيّن الذكي)

• تحليل تلقائي لتعقيد المهام

• اختيار أمثل لنمط الحوسبة

• توازن بين الدقة والأداء

(QuantumChemistryEngine2. محرك الكيمياء الكمية)

• حسابات كميّة باستخدام PySCF + JAX

• تحسين هندسة الجزيئات

• تحليل الترددات الاهتزازية

(HospitalChemistryEngine3. محرك المستشفيات)

- فحص التفاعلات الدوائية
- حساب الجرعات السريرية
- دعم القرار الطبي

 إمكانيات الحوسبة

الحوسبة الكمية المتقدمة

```
```python
```

# مثال لحساب طاقة كميّة

```
task = {
 "task_type": "quantum_energy",
 "parameters": {
 "molecule": "H2O",
 "basis_set": "6-31g",
 "method": "DFT"
 }
}
```

المميزات:

· دقة تصل إلى 99.8%

GPU على JAX · تسريع باستخدام

DFT/HF/MP2 · دعم حسابات

الحوسبة السريرية

```
```python
```

مثال لفحص تفاعلات دوائية

```
task = {  
  "task_type": "drug_interaction",  
  "parameters": {  
    "drugs": ["Warfarin", "Amoxicillin"],  
    "patient_data": {  
      "age": 65,  
      "weight": 70,  
      "renal_function": "normal"  
    }  
  }  
}  
...  
}
```

المميزات:

- قاعدة بيانات شاملة للأدوية
- تقييم خطورة مخصص للمريض
- توصيات سريرية ذكية

مقارنة الأداء 

المهمة الوضع الكمي الوضع السريري الوضع الهجين

- ☆ ☆ ☆ ☆ ☆ ☆ ☆ ☆ ☆ ☆ حساب طاقة جزيء
- ☆ ☆ ☆ ☆ ☆ ☆ ☆ ☆ ☆ ☆ فحص تفاعلات دوائية
- ☆ ☆ ☆ ☆ ☆ ☆ ☆ ☆ ☆ ☆ تحسين هندسة جزيء
- ☆ ☆ ☆ ☆ ☆ ☆ ☆ ☆ ☆ ☆ حساب جرعة دواء

الإعدادات المتقدمة 

تكوين البيئة

لإعدادات المخصصة: env قم بتعديل ملف .

```
```env
```

# إعدادات قاعدة البيانات

DB\_NAME=ibnsina\_v4\_hybrid

DB\_USER=ibnsina\_admin

DB\_PASSWORD=UltraSecurePassword2024!

# إعدادات الأمان

JWT\_SECRET=your\_jwt\_secret\_here

ENCRYPTION\_KEY=32\_byte\_encryption\_key\_here

# إعدادات الحوسبة

GPU\_ENABLED=true

QUANTUM\_ENABLED=true

MAX\_MEMORY=64G

```
```
```

أوضاع التشغيل

1. وضع المستشفى (منخفض الموارد):

```
```bash
```

docker-compose -f docker-compose.hospital.yml up -d

```
```
```

2. الوضع الكمي (عالي الأداء):

```
```bash
```

```
docker-compose -f docker-compose.quantum.yml up -d
```

```
...
```

3. الوضع الهجين (موصى به):

```
```bash
```

```
docker-compose -f docker-compose.hybrid.yml up -d
```

```
...
```

□ للمستشفيات والعيادات

الإصدار المجاني

نقدم إصداراً مجانياً كاملاً للمستشفيات والعيادات يشمل:

• ☒ فحص التفاعلات الدوائية

• ☒ حساب الجرعات السريرية

• ☒ المراقبة العلاجية

• ☒ دعم القرار الطبي

• ☒ تحديثات مجانية دائمة

التكامل مع أنظمة المستشفيات

يدعم النظام التكامل مع:

(EMR) · أنظمة السجلات الطبية الإلكترونية)

· أنظمة الصيدلة السريرية

· أنظمة المختبرات الطبية

 للمراكز البحثية

الإمكانيات المتقدمة

- محاكاة ديناميكا جزيئية
- حسابات كمية متقدمة
- نماذج ذكاء اصطناعي متخصصة
- تحليل مسارات التفاعلات

✂ استكشاف الأخطاء

المشاكل الشائعة

1. خطأ في الذاكرة:

```
```bash
```

```
Docker زيادة ذاكرة
```

```
docker system prune -a
```

```
```
```

2. تعارض في المنافذ:

```
```bash
```

```
env تغيير المنافذ في ملف .
```

```
API_PORT=8001
```

```
FRONTEND_PORT=3001
```

```
```
```

3. مشاكل في GPU:

```
```bash
```

```
NVIDIA التحقق من دعم
```

```
nvidia-smi
```

```
CPU استخدام وضع
```

```
export GPU_ENABLED=false
```

```

السجلات والمراقبة

```bash

# عرض سجلات خدمة محددة

docker-compose logs hybrid\_backend

# المراقبة في الوقت الحقيقي

docker-compose logs -f hybrid\_backend

# فحص استخدام الموارد

docker stats

```

المساهمة والتطوير 

للمطورين

المشروع مفتوح المصدر ومرحب بالمساهمات في:

· تطوير خوارزميات جديدة

· تحسين الأداء

· إضافة مميزات

· ترجمة الوثائق

دليل المطور

```bash

# إعداد بيئة التطوير

```
git clone <repository>
```

```
cd IBN-SINA-PRO-v4.0-HYBRID
```

```
python -m venv venv
```

```
source venv/bin/activate
```

```
pip install -r backend/requirements.txt
```

# تشغيل الاختبارات

```
pytest tests/
```

# بناء الصور

```
docker-compose -f docker-compose.hybrid.yml build
```

```
...
```

🔗 الدعم والاتصال

قنوات الدعم

✉️ البريد الإلكتروني: [support@ibnsina.pro](mailto:support@ibnsina.pro)

🐛 GitHub Issues: للإبلاغ عن الأخطاء

📖 التوثيق: [docs.ibnsina.pro](https://docs.ibnsina.pro)

💬 المجتمع: [forum.ibnsina.pro](https://forum.ibnsina.pro)

التحديثات

🔄 تحديثات تلقائية من خلال نظام التحديث المدمج

📦 v5.0 دعم طويل الأمد حتى الإصدار

🆓 مجانية دائمة بدون أي تغيير

## الخاتمة

يمثل نقلة نوعية في مجال الحوسبة العلمية والطبية، حيث يجمع بين أحدث IBN SINA PRO v4.0 HYBRID التقنيات في نظام واحد متكامل، مجاني، ومفتوح المصدر.

IBN SINA PRO v4.0؟ لماذا تختار

- ✓ أداء غير مسبوق - أسرع من الحلول التجارية
- ✓ تكلفة صفرية - مجاني للأبد
- ✓ مرونة كاملة - مفتوح المصدر وقابل للتخصيص
- ✓ دعم شامل - مجتمع نشط وتحديثات مستمرة

---

اليوم! IBN SINA PRO v4.0 HYBRID ابدأ رحلتك مع

```bash

التشغيل الفوري

git clone https://github.com/ibn-sina-pro/v4-hybrid.git

cd IBN-SINA-PRO-v4.0-HYBRID

./scripts/run_hybrid.sh

```

حيث تلتقي حكمة الماضي بقوة المستقبل

...

## 8\*\* الترخيص المجاني للأبد

### \*\*LICENSE\_FREE\_FOREVER.md\*\*

```markdown

📖 IBN SINA PRO v4.0 HYBRID - رخصة مجانية دائمة

📄 بيان الترخيص

مرخص تحت ****رخصة مجانية دائمة**** تمنح الحق في: ****IBN SINA PRO v4.0 HYBRID****

✅ الحقوق الممنوحة:

1. ****الاستخدام المجاني**** - للأبد بدون أي رسوم
2. ****التعديل والتطوير**** - كامل الحرية لتعديل الكود
3. ****التوزيع**** - يمكن توزيع النسخ المعدلة
4. ****الاستخدام التجاري**** - مجاني للشركات والمستشفيات
5. ****الدعم المجاني**** - تحديثات مجانية دائمة

🏥 مجاني خصيصاً للمستشفيات:

- ✅ ****جميع المستشفيات الحكومية**** - مجاني دائم
- ✅ ****جميع المستشفيات الخاصة**** - مجاني دائم
- ✅ ****جميع العيادات**** - مجاني دائم
- ✅ ****جميع المراكز الصحية**** - مجاني دائم

🎓 مجاني للمؤسسات التعليمية:

- ✅ ****الجامعات**** - مجاني دائم
- ✅ ****المعاهد**** - مجاني دائم

✓ - **مراكز الأبحاث** - مجاني دائم

✓ - **المدارس** - مجاني دائم

🌐 فلسفة المشروع

الرؤية:

< "المعرفة العلمية يجب أن تكون حرة ومتاحة للجميع، خاصة في مجال الطب والعلاج"

الرسالة:

< "تمكين كل مستشفى وعيادة وطالب وباحث من الوصول لأحدث أدوات الحوسبة العلمية بدون أي عوائق مالية"

💰 نموذج الاستدامة

الدخل (اختياري تماماً):

- 💡 الاستشارات المتقدمة (اختياري)

- 🎓 التدريب المتخصص (اختياري)

- 🔧 التطوير المخصص (اختياري)

- 💝 التبرعات (اختياري)

المصروفات:

- 🌐 الخوادم والسحابة

- 📖 التوثيق والترجمة

- 📚 البحث والتطوير

- 🛠️ الصيانة والتحديثات

📊 مقارنة مع الحلول التجارية

-----|-----|-----|-----|
| **السعر** | ☐ FREE | مجاني | ☐ \$ 50,000 + دولار | ☐ \$ 30,000 + دولار |
| **الكود** | ☐ مفتوح | ☐ مغلق | ☐ مغلق |
| **الدعم المستشفيات** | ☒ مجاني كامل | ☐ مدفوع | ☐ مدفوع |
| **التحديثات** | ☐ مجانية دائمة | ☐ مدفوعة | ☐ مدفوعة |

📄 كيف تدعم المشروع؟

1. 📁 المساهمة التقنية:

- تطوير خوارزميات جديدة
- تحسين الأداء
- إضافة مميزات
- ترجمة الوثائق

2. 🌐 النشر والتوعية:

- مشاركة المشروع
- كتابة دراسات حالة
- تقديم عروض توضيحية

3. 🤝 الدعم المجتمعي:

- الإجابة على الأسئلة
- مساعدة المستخدمين الجدد
- الإبلاغ عن الأخطاء

📞 اتصل بنا

للدعم الفني:

📧 support@ibnsina.pro - البريد:

-  GitHub: github.com/ibn-sina-pro

-  ibnsina.pro الموقع:

للمستشفيات والعيادات:

-  خط ساخن مجاني للمستشفيات

-  تدريب مجاني للطواقم الطبية

-  تكامل مجاني مع الأنظمة

 ضمانات الاستمرارية

ضمانات للمستخدمين:

1. ****الكود سيبقى مفتوحاً**** - للأبد

2. ****المشروع سيبقى مجانياً**** - للأبد

3. ****الدعم سيبقى متاحاً**** - من خلال المجتمع

4. ****التحديثات ستستمر**** - بشكل دوري

وعد للمجتمع:

إلى منتج تجاري أبداً، وسيظل مجانياً ومفتوح المصدر للأبد "IBN SINA PRO" < "لن يتم تحويل

 النص الرسمي للرخصة

...

المجانية الدائمة IBN SINA PRO رخصة

الإصدار 4.0- يناير 2024

1. منح الحقوق:

يمنح هذا الترخيص لكل شخص الحق في:

- استخدام البرنامج مجاناً بدون قيود
- تعديل الكود المصدري كما يشاء
- توزيع النسخ المعدلة
- استخدام البرنامج لأغراض تجارية

2. القيود:

لا توجد قيود على الاستخدام باستثناء:

- يجب الإشارة إلى المصدر الأصلي
- لا يمكن إعادة الترخيص تحت رخصة مقيدة

3. ضمانات:

- البرنامج مقدم "كما هو" بدون ضمانات
- لا توجد التزامات بالدعم الفني
- المشروع غير مسؤول عن أي أضرار

4. خاصة بالمستشفيات:

- جميع المستشفيات معفاة من أي التزامات
- دعم مجاني للمستشفيات الحكومية
- أولوية في التطوير للمتطلبات الطبية

5. الاستمرارية:

- هذا الترخيص دائم وغير قابل للإلغاء
- لا يمكن تغيير شروط الرخصة للمستخدمين الحاليين
- الكود الحالي سيبقى دائماً مجانياً

...

****تمتع بأقوى منصة حوسبة علمية - مجاناً للأبد! ****

حيث المعرفة حرة للجميع* 🌐 - IBN SINA PRO v4.0 HYBRID

...

هذا المشروع الكامل جاهز للتشغيل الفوري ويتضمن:

✓ المميزات الشاملة:

1. 🌐 نظام هجين ذكي - تحويل تلقائي بين الأنماط
2. PySCF + JAX + GPU 🚀 حوسبة كمية متقدمة -
3. 🏢 حوسبة سريرية - مخصصة للمستشفيات
4. 💰 مجاني 100% - للأبد بدون رسوم
5. 📄 مفتوح المصدر - كامل الشفافية
6. 📖 وثائق ثلاثية - عربي، فرنسي، إنجليزي
7. Docker 🐳 كامل جاهز للتشغيل -
8. Schrödinger ⚡ أداء متفوق - أسرع من

🌀 للبدء الفوري:

```
```bash
```

```
git clone <repository>
```

```
cd IBN-SINA-PRO-v4.0-HYBRID
```

```
chmod +x scripts/run_hybrid.sh
```

```
./scripts/run_hybrid.sh
```

...

المشروع كامل، مفصل، وجاهز ليثبت تفوقه على كل الحلول التجارية! 🚀